

PORTOFOLIO (UAS)

TP – PBO 2025

Implementasi Pemrograman Berorientasi Objek pada Sistem Informasi Service Motor Berbasis Framework Django

a. Identitas Proyek

NIM : 2400018009

Nama : Farhan

Matakuliah : Pemrograman Berorientasi Objek

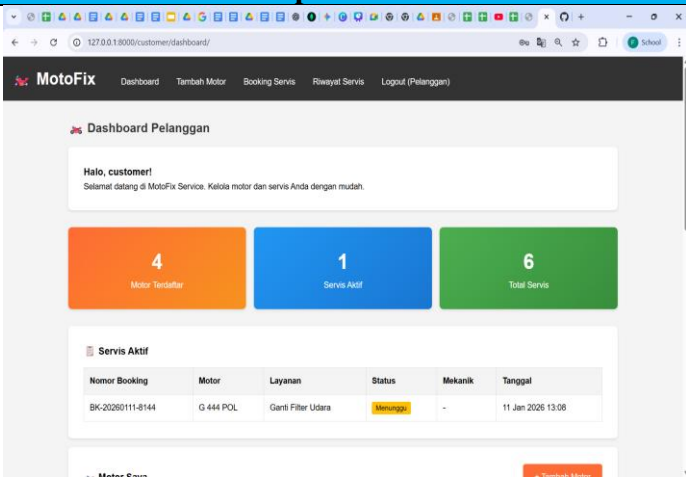
Judul Proyek

Implementasi Pemrograman Berorientasi Objek pada Sistem Informasi Service Motor Berbasis Framework Django

Link Repository

<https://github.com/Revoluz/MotoFix-Web-Based-OOP-Service-Manager-.git>

Capture tampilan awal aplikasi/ tampilan utama

User	Tampilan utama
Pelanggan	

Admin

--	--	--

b. Persoalan Bisnis dan Deskripsi Proyek

Bengkel service motor pada umumnya masih menggunakan sistem pencatatan manual dalam mengelola data pelanggan, data motor, serta proses booking dan servis. Hal tersebut menimbulkan beberapa permasalahan, antara lain:

- Data pelanggan dan motor tidak terdokumentasi dengan baik
- Proses booking servis sering terjadi kesalahan jadwal
- Pelanggan sulit memantau status pengerjaan motor
- Pembuatan invoice masih manual dan rawan kesalahan

Berdasarkan permasalahan tersebut, proyek ini bertujuan untuk membangun Sistem Informasi Service Motor berbasis web menggunakan framework Django dengan pendekatan Pemrograman Berorientasi Objek (OOP). Sistem ini diharapkan mampu membantu pengelolaan servis motor secara terstruktur, efisien, dan mudah digunakan oleh pelanggan maupun admin.

c. Daftar Seluruh spesifikasi aplikasi

1. Spesifikasi Pengguna (User)

Aplikasi memiliki dua jenis pengguna:

- Pelanggan
- Admin

2. Fitur Pelanggan

- Login dan logout pelanggan
- Menambahkan dan mengelola data motor
- Memilih layanan servis
- Melakukan booking servis (booking / walk-in)

- Melihat status pengerjaan servis
- Melihat dan menerima invoice digital

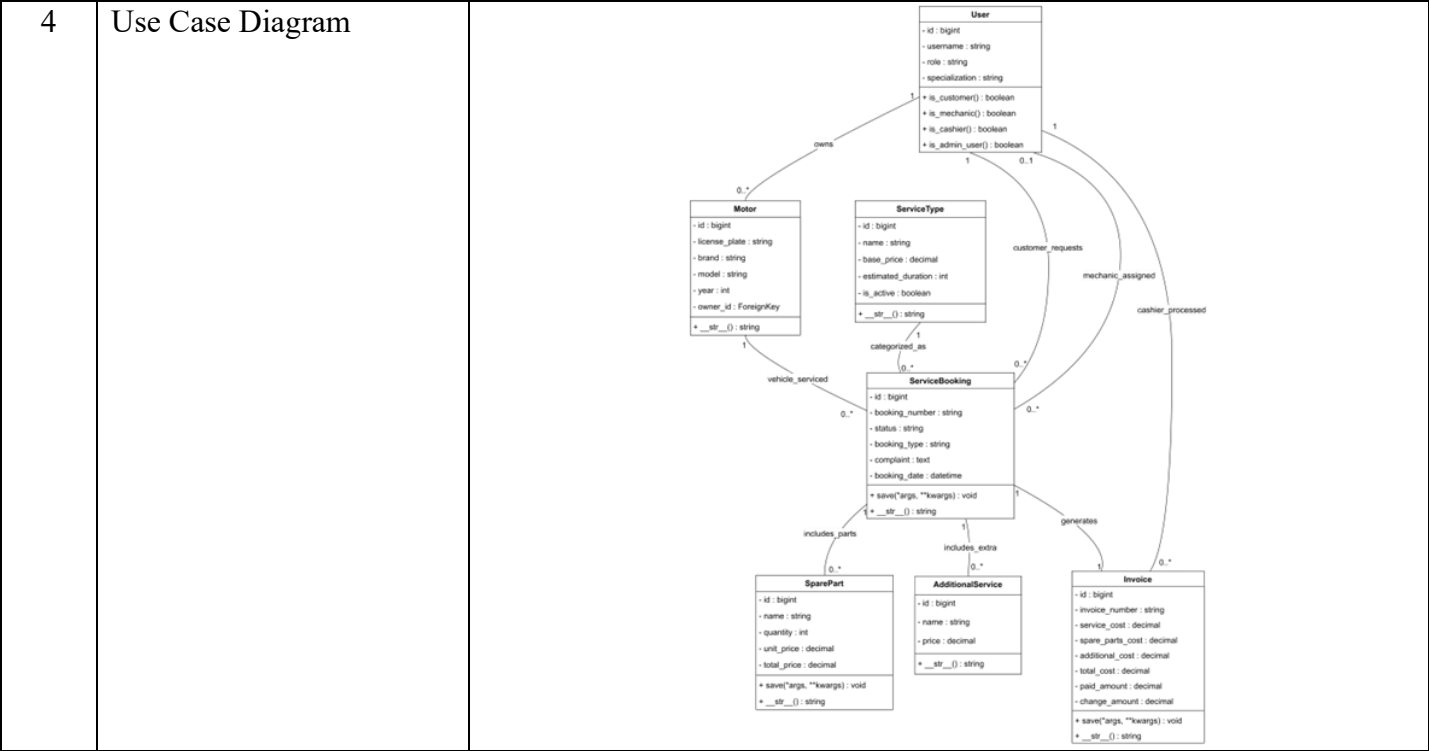
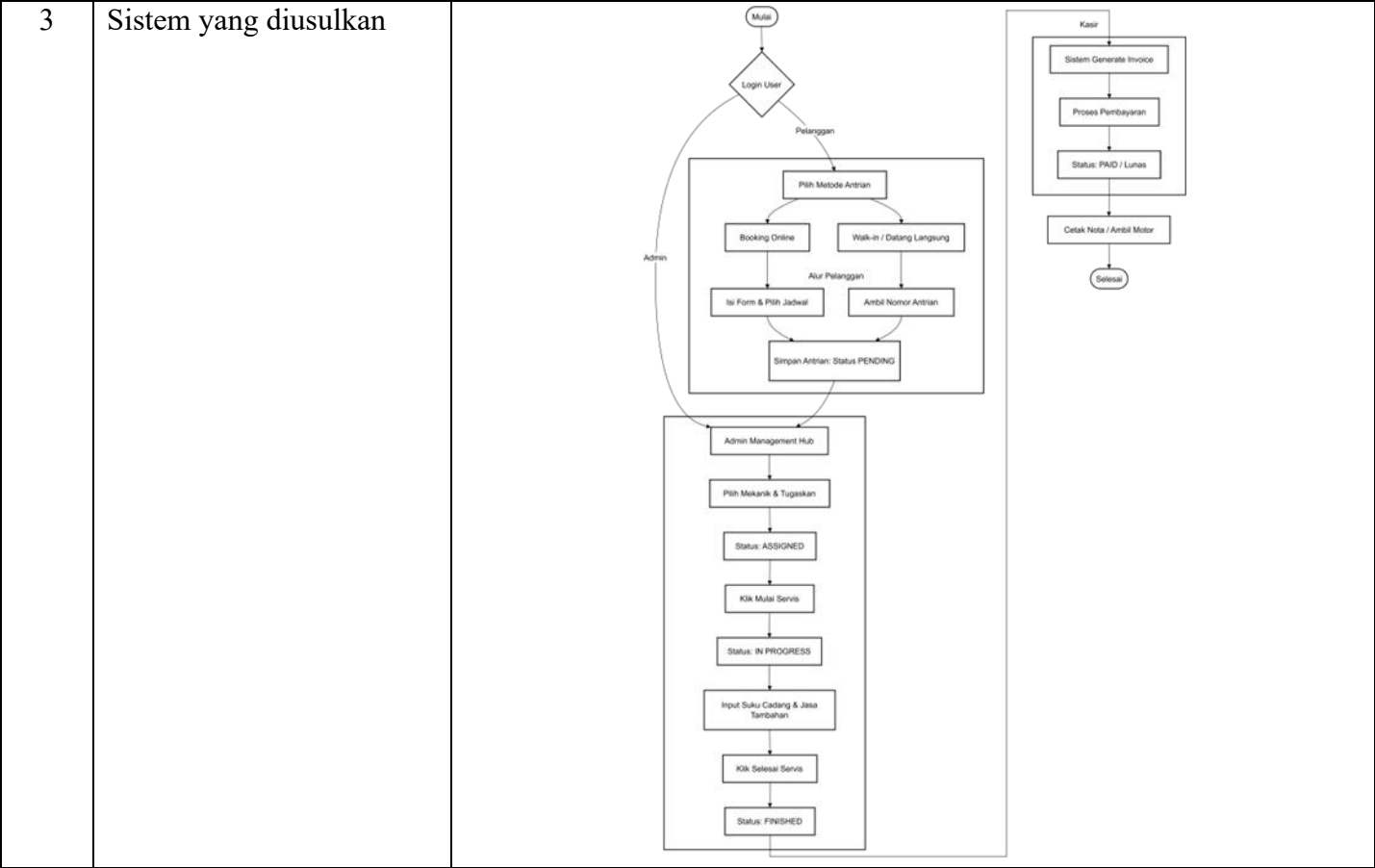
3. Fitur Admin

- Login dan logout admin
- Melihat daftar booking servis
- Mengelola data layanan servis
- Mengelola data mekanik
- Menugaskan mekanik ke servis tertentu
- Mengelola status servis
- Mengirim invoice digital ke pelanggan

d. Rancangan Model Diagram UML

Rancangan UML yang digunakan pada proyek ini meliputi:

No	Rancangan	Diagram UML
1	Kerangka sistem informasi	
2	Sistem berjalan	

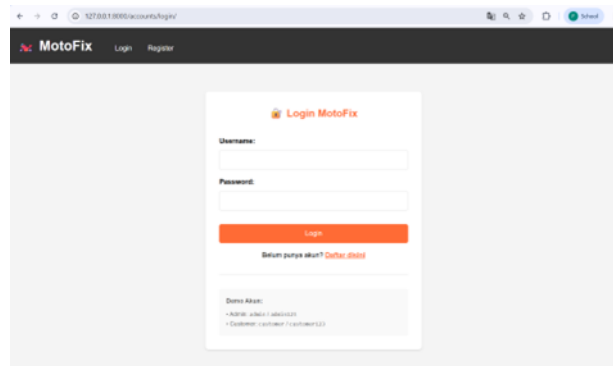


e. **Rancangan Antar Muka berbasis GUI**

Antarmuka aplikasi dirancang berbasis web menggunakan HTML, CSS (Bootstrap) dengan fokus pada kemudahan penggunaan. Rancangan GUI dibedakan berdasarkan peran pengguna:

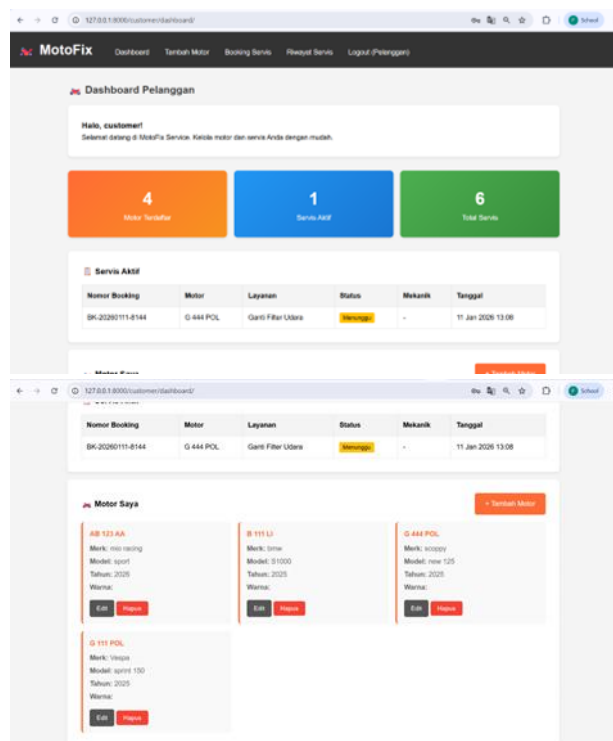
1. GUI Pelanggan

- Halaman Login :



Gambar Halaman Form Login Pelanggan

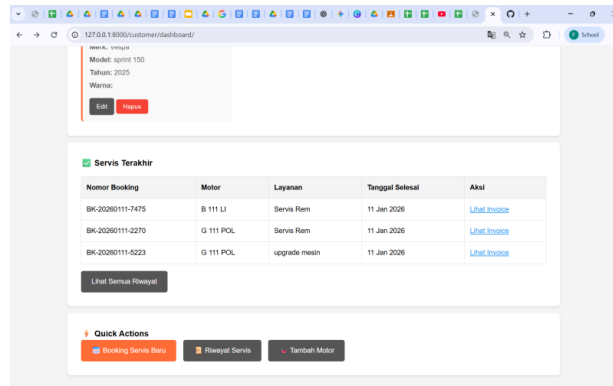
- Dashboard Pelanggan :



Nomor Booking	Motor	Layanan	Status	Mekanik	Tanggal
BK-2020111-8144	G 444 POL	Ganti Filter Udara	Selesai	-	11 Jan 2020 13:08

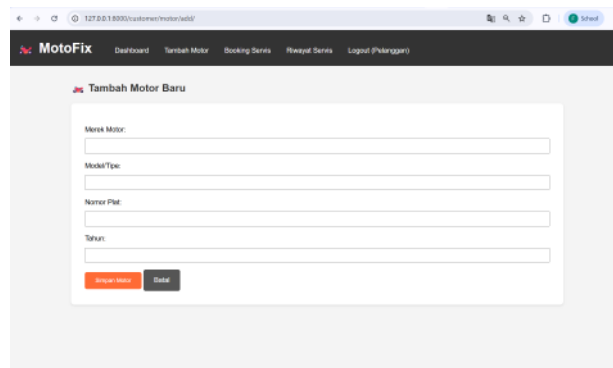
Nomor Booking	Motor	Layanan	Status	Mekanik	Tanggal
BK-2020111-8144	G 444 POL	Ganti Filter Udara	Selesai	-	11 Jan 2020 13:08

Motor Saya	Motor Saya	Motor Saya
A8 123 AB Mark: m888g Model: sport Tahun: 2020 Warna: Edi Hasan	B 111 L2 Mark: m888g Model: G 1000 Tahun: 2020 Warna: Edi Hasan	G 444 POL Mark: m888g Model: m888g Tahun: 2020 Warna: Edi Hasan
G 111 POL Mark: m888g Model: sport 150 Tahun: 2020 Warna: Edi Hasan		



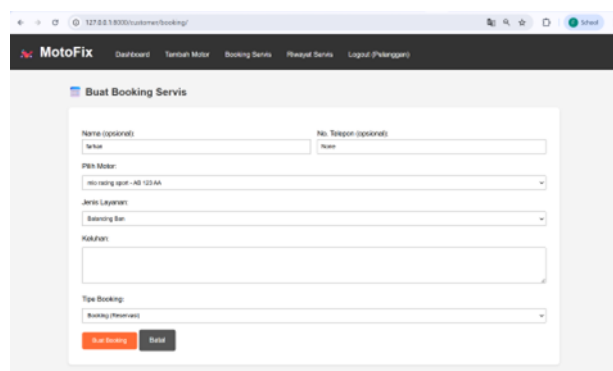
Gambar Halaman Dashboard Pelanggan

- Halaman Tambah Motor :



Gambar Halaman Tambah Motor Pelanggan

- Halaman booking servis motor pelanggan :



Gambar halaman booking servis motor pelanggan

- Halaman Riwayat servis :

Nomor Booking	Tanggal	Motor	Layanan	Status	Aksi
BK-20260114-0905	14 Jan 2026 11:04	AB 330204 CD Honda Beat	Cuci Motor	Sudah Dikerjakan	Ulang Booking
BK-20260114-0902	14 Jan 2026 11:43	AB 330204 CD Honda Beat	Ganti Ban	Sudah Dikerjakan	Ulang Booking
BK-20260114-0901	14 Jan 2026 11:43	AB 330204 CD Honda Beat	raf	Sudah Dikerjakan	Ulang Booking
BK-20251230-7631	30 Des 2025 16:19	AB 330204 CD Honda Beat	Ganti Kampas Rem	Sudah Dikerjakan	Ulang Booking
BK-20251230-1463	30 Des 2025 16:19	AB 330204 CD Honda Beat	Cuci Motor	Sudah Dikerjakan	Ulang Booking
BK-20251230-4569	30 Des 2025 15:54	AB 330204 CD Honda Beat	Cuci Motor	Sudah Dikerjakan	Ulang Booking

Gambar halaman Riwayat servis motor pelanggan

- Halaman Cetak Invoice

1/19/26, 8:03 PM

Invoice Detail - MotoFix

Invoice Servis #INV-BK-20260111-7475

Informasi Servis

Nomor Booking: BK-20260111-7475

Tanggal Booking: 11 Januari 2026 22:02

Tanggal Selesai: 11 Januari 2026 22:05

Motor: bmw S1000 - B 111 LI

Mekanik: Bapak Roy

Layanan: Servis Rem

Status: Sudah Dibayar

Detail Biaya

Item	Jumlah
Biaya Jasa Servis (Servis Rem)	Rp 75000
TOTAL:	Rp 75000
Dibayar (Tunai)	Rp 75000
Kembalian	Rp 0

Kasir:

Tanggal Invoice: 19 Januari 2026 14:24

Gambar halaman pelanggan cetak invoice

2. GUI Admin

- Halaman Login :

127.0.0.1:8000/accounts/login/

MotoFix Login Register

Login MotoFix

Username:

Password:

Login

Belum punya akun? [Daftar disini](#)

Demo Akun:

- Admin: admin123456789
- Customer: customer123456789123

Gambar halaman form login Admin

- Dashboard Admin :

127.0.0.1:8000/admin-hub/bookings/

MotoFix Dashboard Kelola User Kelola Servis Mekanik Logout (Admin)

Manajemen Booking Service

6 Total Booking

1 Menunggu

0 Digesek

0 Dierjakan

0 Selesai

MotoFix Dashboard Tambah Motor Booking Servis Riwayat Servis Logout (Penyagar)

Riwayat Servis

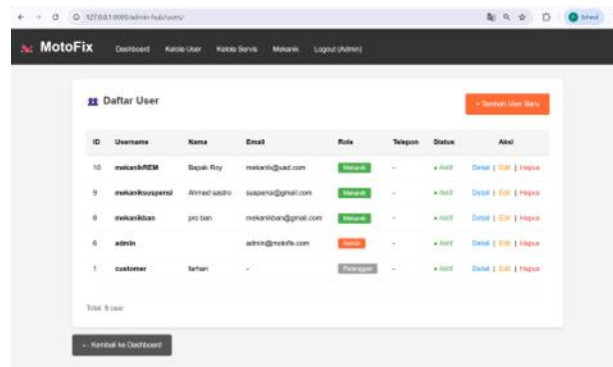
Filter Status: Service Status Filter

Nomor Booking	Tanggal	Motor	Layanan	Status	Aksi
BK-20250114-0905	14 Jan 2025 11:34	AB 330204 CD Honda Beat	Cuci Motor	Booked	View Booking
BK-20250114-0902	14 Jan 2025 11:03	AB 330204 CD Honda Beat	Ganti Ban	Booked	View Booking
BK-20250114-0901	14 Jan 2025 11:00	AB 330204 CD Honda Beat	Sal	Booked	View Booking
BK-20251230-7631	30 Dec 2025 10:10	AB 330204 CD Honda Beat	Ganti Kampas Rem	Booked	View Booking
BK-20251230-1463	30 Dec 2025 10:10	AB 330204 CD Honda Beat	Cuci Motor	Booked	View Booking
BK-20251230-4569	30 Dec 2025 10:10	AB 330204 CD Honda Beat	Cuci Motor	Menunggu	View Booking

[Kembali ke Dashboard](#)

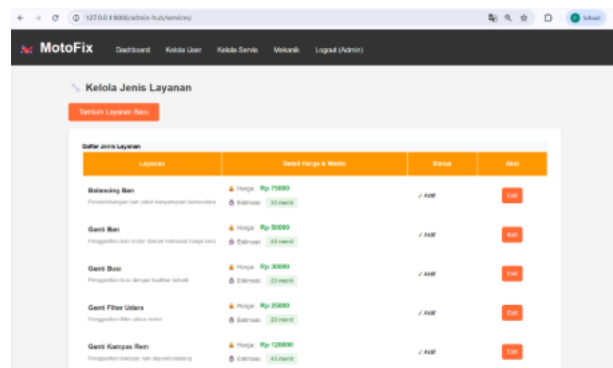
Gambar halaman Dashboard Admin

- Halaman Admin Kelola user :



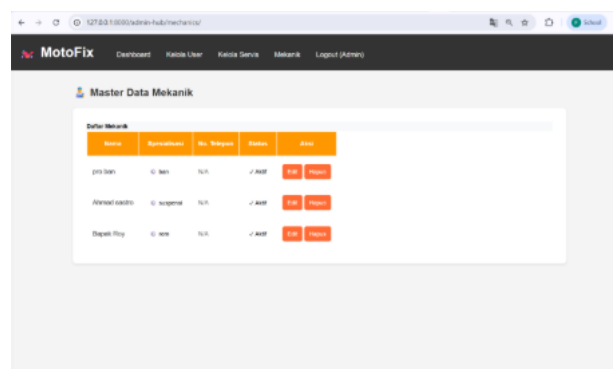
Gambar halaman admin Kelola user

- Halaman admin Kelola layanan servis :



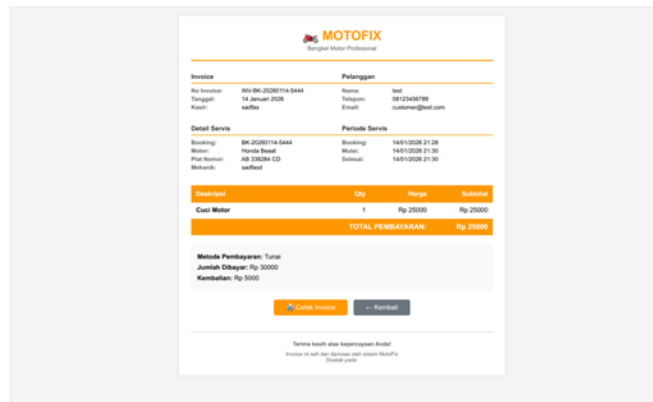
Gambar halaman admin Kelola layanan servis

- Halaman admin Kelola mekanik :



Gambar halaman admin Kelola mekanik

- Halaman invoice (admin mencetak invoice) :



Gambar halaman admin cetak invoice

f. Skrip Program dan penjelasannya

1. Login

Fitur login pada sistem MotoFix digunakan untuk melakukan autentikasi pengguna sehingga sistem dapat mengenali identitas dan peran (role) pengguna. Proses login ini memanfaatkan sistem autentikasi bawaan Django dengan menggunakan Custom User Model.

- Model User sebagai Dasar Login

File : accounts/models.py

```
from django.db import models
from django.contrib.auth.models import AbstractUser

# Create your models here.

class User(AbstractUser):
    """
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='customer')
```

Penjelasan :

- AbstractUser menyediakan field bawaan: username, password, is_active, last_login.
- Dengan mewarisi AbstractUser, sistem login Django tetap digunakan tanpa membuat autentikasi dari awal.
- Field role digunakan setelah login berhasil untuk menentukan hak akses dan dashboard pengguna.

Bagian ini menjadi dasar karena semua proses login selalu menggunakan data dari model User

- Migration Pembuatan Tabel User
File : accounts/migrations/001_initial.py

```
class Migration(migrations.Migration):

    initial = True

    dependencies = [
        ('auth', '0012_alter_user_first_name_max_length'),
    ]

    operations = [
        migrations.CreateModel(
            name='User',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('password', models.CharField(max_length=128, verbose_name='password')),
                ('last_login', models.DateTimeField(blank=True, null=True, verbose_name='last login')),
                ('is_superuser', models.BooleanField(default=False, help_text='Designates that this user has all permissions without explicit permission assignment.', verbose_name='Is superuser')),
                ('username', models.CharField(error_messages={'unique': 'A user with that username already exists.'}, help_text='Required. 150 characters or fewer. Letters, digits and hyphens only.', max_length=150, verbose_name='username')),
                ('first_name', models.CharField(blank=True, max_length=150, verbose_name='first name')),
                ('last_name', models.CharField(blank=True, max_length=150, verbose_name='last name')),
                ('email', models.EmailField(blank=True, max_length=254, verbose_name='email address')),
                ('is_staff', models.BooleanField(default=False, help_text='Designates whether the user can log into this admin site.', verbose_name='Is staff')),
                ('is_active', models.BooleanField(default=True, help_text='Designates whether this user should be treated as active. Unselect this checkbox if you want to deactivate the user. Use the is_active manager method to actually change the active state.', verbose_name='Is active')),
                ('date_joined', models.DateTimeField(default=django.utils.timezone.now, verbose_name='date joined')),
                ('role', models.CharField(choices=[('customer', 'Pelanggan'), ('mechanic', 'Mekanik'), ('cashier', 'Kasir'), ('admin', 'Admin')], max_length=15, verbose_name='role')),
                ('phone', models.CharField(blank=True, max_length=15, null=True)),
                ('address', models.TextField(blank=True, null=True)),
                ('created_at', models.DateTimeField(auto_now_add=True)),
                ('updated_at', models.DateTimeField(auto_now=True)),
                ('groups', models.ManyToManyField(blank=True, help_text='The groups this user belongs to. A user will get all permissions granted to each of their groups.', related_name='user_groups', to='auth.Group', verbose_name='groups')),
                ('user_permissions', models.ManyToManyField(blank=True, help_text='Specific permissions for this user.', related_name='user_permissions', to='auth.Permission', verbose_name='user permissions')),
            ],
            options={
                'verbose_name': 'User',
                'verbose_name_plural': 'Users',
                'db_table': 'users',
            },
            managers=[
                ('objects', django.contrib.auth.models.UserManager()),
            ],
        ),
    ]
```

Penjelasan:

- Migration ini membuat tabel users di database.
- Field username digunakan untuk pencarian akun saat login.
- Field password menyimpan password dalam bentuk hash.
- Field is_active menentukan apakah akun boleh login.
- Field last_login otomatis diperbarui saat login berhasil.

Tanpa tabel ini, proses login tidak dapat dilakukan karena tidak ada data akun yang bisa diverifikasi.

- Pembuatan Akun Awal (Admin & Customer)
File: setup_admin.py / setup_demo_users.py

```

from accounts.models import User

print("🔪 Membuat superuser admin untuk MotoFix...")
print("=" * 50)

# Hapus admin jika sudah ada
User.objects.filter(username='admin').delete()

# Buat superuser admin
admin = User.objects.create_superuser(
    username='admin',
    email='admin@motofix.com',
    password='admin123',
    role='admin'
)
print(f"✓ Superuser Admin dibuat: {admin.username}")

print("=" * 50)
print("✅ Superuser berhasil dibuat!")
print("")
print("📄 Admin Login:")
print("-" * 50)
print("| Username | Password |")
print("-" * 50)
print("| admin    | admin123 |")
print("-" * 50)
print("")
print("🔑 Login di: http://127.0.0.1:8000/accounts/login/)")
print("🔒 Admin Panel: http://127.0.0.1:8000/admin/)")

```

```

1 # Script untuk membuat user demo
2 # Jalankan dengan: python manage.py shell < setup_demo_users.py
3
4 from accounts.models import User
5
6 print("\n Membuat user demo untuk MotoFix...")
7 print("=" * 50)
8
9 # Hapus user demo jika sudah ada
10 User.objects.filter(username__in=['customer', 'mekanik', 'kasir']).delete()
11
12 # Buat user customer
13 customer = User.objects.create_user(
14     username='customer',
15     email='customer@test.com',
16     password='customer123',
17     role='customer',
18     phone='08123456789',
19     address='Jl. Pelanggan No. 123, Jakarta'
20 )
21 print(f"✓ User Customer dibuat: {customer.username}")
22
23 # Buat user mekanik
24 mekanik = User.objects.create_user(
25     username='mekanik',
26     email='mekanik@test.com',
27     password='mekanik123',
28     role='mechanic',
29     phone='08234567890',
30     address='Jl. Mekanik No. 456, Jakarta'
31 )
32 print(f"✓ User Mekanik dibuat: {mekanik.username}")
33
34 # Buat user kasir
35 kasir = User.objects.create_user(
36     username='kasir',
37     email='kasir@test.com',
38     password='kasir123',
39     role='cashier',
40     phone='08345678901',
41     address='Jl. Kasir No. 789, Jakarta'
42 )
43 print(f"✓ User Kasir dibuat: {kasir.username}")

```

Penjelasan :

- create_superuser() membuat akun admin dengan hak penuh.
- create_user() membuat akun customer biasa.
- Password otomatis dienkripsi oleh Django.

Akun ini disimpan di tabel users dan digunakan sebagai data awal untuk proses login.

- URL Login (Entry Point Login)
File: accounts/urls.py

```

from django.urls import path
from . import views

app_name = 'accounts'

urlpatterns = [
    # Autentikasi & Akun
    path('login/', views.login, name='login'),
    path('logout/', views.logout, name='logout'),
    path('register/', views.register, name='register'),
]

```

Penjelasan :

- o Menentukan alamat URL /accounts/login/.
- o Setiap request ke URL ini akan diteruskan ke fungsi login() di file views.py.

- View Login (Inti Proses Autentikasi)

File accounts/views.py

a. Mengambil input user

```

# Autentikasi & Akun
def login(request):
    """Halaman login untuk semua user"""
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        user = authenticate(request, username=username, password=password)

        if user is not None:
            auth_login(request, user)
            messages.success(request, f'Selamat datang, {user.username}!')

            # Redirect berdasarkan role
            if user.is_admin_user or user.is_cashier or user.is_mechanic:
                return redirect('admin_hub:admin_dashboard')
            else:
                return redirect('customer:customer_dashboard')
        else:
            messages.error(request, 'Username atau password salah!')

    return render(request, 'accounts/login.html')

```

b. Autentikasi

user = authenticate(request, username=username, password=password)

Penjelasan :

- o Django mencari data user di tabel users.
- o Password dibandingkan dalam bentuk hash.
- o Jika cocok, variabel user berisi objek User.

c. Login berhasil

```
if user is not None:
    auth_login(request, user)
    messages.success(request, f'Selamat datang, {user.username}!')

    # Redirect berdasarkan role
    if user.is_admin_user or user.is_cashier or user.is_mechanic:
        return redirect('admin_hub:admin_dashboard')
    else:
        return redirect('customer:customer_dashboard')
else:
    messages.error(request, 'Username atau password salah!')

return render(request, 'accounts/login.html')

auth_login(request, user)
messages.success(request, f'Selamat datang, {user.username}!')
```

Penjelasan :

- Membuat session login.
- Menandai bahwa user telah terautentikasi.

d. Redirect berdasarkan role

```
# Redirect berdasarkan role
if user.is_admin_user or user.is_cashier or user.is_mechanic:
    return redirect('admin_hub:admin_dashboard')
else:
    return redirect('customer:customer_dashboard')
else:
    messages.error(request, 'Username atau password salah!')
```

Penjelasan:

- Admin diarahkan ke dashboard admin.
- Customer diarahkan ke dashboard customer.

e. Template Login (Form Input User)

File: templates/accounts/login.html

```
<form method="POST">
{% csrf_token %}
<input type="text" name="username">
<input type="password" name="password">
<button type="submit">Login</button>
</form>
```

Penjelasan :

- User memasukkan username dan password.
- Data dikirim ke views.login menggunakan metode POST.

f. Session Login

Setelah fungsi `auth_login()` dijalankan:

- Django membuat session login.
- `request.user.is_authenticated` bernilai True.
- User dapat mengakses halaman yang dilindungi.

g. Kesimpulan

- User membuka halaman login
- Input username & password
- URL diteruskan ke `views.login`
- `authenticate()` mencocokkan data ke tabel users
- `auth_login()` membuat session
- Sistem mengecek roleUser diarahkan ke dashboard sesuai peran

2. Halaman customer Dashboard

Fitur dashboard pelanggan pada sistem MotoFix digunakan sebagai halaman utama setelah pengguna dengan peran customer berhasil melakukan login. Dashboard ini berfungsi untuk menampilkan informasi penting terkait data motor, status servis yang sedang berjalan, serta riwayat servis pelanggan. Seluruh proses pengambilan dan penampilan data dilakukan secara terkontrol berdasarkan akun yang sedang login.

- Proteksi Akses Dashboard Pelanggan

File: `accounts/views.py`

```
# Decorator untuk cek role
def customer_required(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if not request.user.is_customer:
            messages.error(request, 'Akses ditolak! Halaman ini khusus untuk pelanggan.')
            return redirect('admin_hub:admin_dashboard')
        return view_func(request, *args, **kwargs)
    return wrapper
```

Penjelasan :

- Decorator `customer_required` digunakan untuk membatasi akses halaman dashboard.
- Jika user belum login, sistem mengarahkan ke halaman login.
- Jika user bukan customer, sistem menolak akses dan mengarahkan ke dashboard admin.
- Dengan mekanisme ini, hanya user dengan peran customer yang dapat mengakses dashboard pelanggan.

- URL Dashboard Pelanggan

File: customer/urls.py

```
from django.urls import path
from . import views

app_name = 'customer'

urlpatterns = [
    # Panel Pelanggan (Customer)
    path('dashboard/', views.customer_dashboard, name='customer_dashboard'),
    path('motor/add/', views.motor_add, name='motor_add'),
    path('motor/<int:pk>/edit/', views.motor_edit, name='motor_edit'),
    path('motor/<int:pk>/delete/', views.motor_delete, name='motor_delete'),
    path('booking/', views.booking_create, name='booking_create'),
    path('history/', views.service_history, name='service_history'),
    path('invoice/<int:pk>/', views.invoice_detail, name='invoice_detail'),
]
```

Penjelasan :

- o URL ini menentukan alamat halaman dashboard pelanggan yaitu /customer/dashboard/.
- o Request ke URL ini akan diteruskan ke fungsi customer_dashboard di file views.py.

- Fungsi Dashboard Pelanggan (customer_dashboard)

File: customer/views.py

```
# Panel Pelanggan (Customer)
@customer_required
def customer_dashboard(request):
    """dashboard utama & tracking status aktif"""
    # Get customer's motors
    motors = Motor.objects.filter(owner=request.user)

    # Get active bookings (not paid or cancelled)
    active_bookings = ServiceBooking.objects.filter(
        customer=request.user
    ).exclude(
        status__in=['paid', 'cancelled']
    ).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')[:5]

    # Get recent completed services
    completed_services = ServiceBooking.objects.filter(
        customer=request.user,
        status__in=['paid', 'finished']
    ).select_related('motor', 'service_type').order_by('-finish_date')[:3]

    context = {
        'motors': motors,
        'motors_count': motors.count(),
        'active_bookings': active_bookings,
        'active_bookings_count': active_bookings.count(),
        'completed_services': completed_services,
        'total_services': ServiceBooking.objects.filter(customer=request.user).count(),
    }

    return render(request, 'customer/dashboard.html', context)
```

Penjelasan :

- Fungsi ini merupakan inti dari fitur dashboard pelanggan.
- Decorator `@customer_required` memastikan hanya customer yang dapat menjalankan fungsi ini.
- Pengambilan Data Motor Pelanggan :

```
# Get customer's motors
motors = Motor.objects.filter(owner=request.user)
```

Penjelasan:

- Mengambil seluruh data motor yang dimiliki oleh user yang sedang login.
- Filter `owner=request.user` memastikan bahwa pelanggan hanya dapat melihat data motornya sendiri.
- Pengambilan Data Booking Servis Aktif

```
# Get active bookings (not paid or cancelled)
active_bookings = ServiceBooking.objects.filter(
    customer=request.user
).exclude(
    status__in=['paid', 'cancelled']
).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')[:5]
```

Penjelasan :

- Mengambil data booking servis milik pelanggan yang belum selesai atau belum dibatalkan.
- Digunakan untuk menampilkan status servis yang sedang berjalan pada dashboard.
- Pengambilan Riwayat Servis Selesai

```
# Get recent completed services
completed_services = ServiceBooking.objects.filter(
    customer=request.user,
    status__in=['paid', 'finished']
).select_related('motor', 'service_type').order_by('-finish_date')[:3]
```

Penjelasan:

- Mengambil data servis yang sudah selesai atau sudah dibayar.
- Digunakan untuk menampilkan riwayat servis pelanggan.

- Kirim data ke template

```
context = {
    'motors': motors,
    'motors_count': motors.count(),
    'active_bookings': active_bookings,
    'active_bookings_count': active_bookings.count(),
    'completed_services': completed_services,
    'total_services': ServiceBooking.objects.filter(customer=request.user).count(),
}
```

Penjelasan :

- o Mengirim semua data ke halaman HTML
 - o Agar bisa ditampilkan ke user
 - o motors_count menampilkan jumlah motor pelanggan.
 - o active_bookings_count menampilkan jumlah servis yang sedang berjalan.
 - o total_services menampilkan total servis yang pernah dilakukan pelanggan.
 - o Seluruh data dikirim ke template untuk ditampilkan dalam bentuk statistik dan daftar informasi.
- Menampilkan Halaman Dashboard

```
return render(request, 'customer/dashboard.html', context)
```

Penjelasan :

- o Menampilkan halaman dashboard.html dengan data yang telah diproses.
 - o Template ini berfungsi sebagai tampilan antarmuka (GUI) bagi pelanggan.
- Template Dashboard Pelanggan = File: templates/customer/dashboard.html
 - o Contoh penggunaan data di template:
 - {{ motors_count }}
 - {{ active_bookings_count }}
 - {{ total_services }}

3. Halaman customer tambah motor

Fitur tambah motor digunakan oleh pelanggan untuk mendaftarkan data kendaraan yang akan diservis di bengkel MotoFix. Data motor ini menjadi dasar dalam proses booking servis, pencatatan riwayat servis, serta pembuatan invoice. Setiap motor dikaitkan langsung dengan akun pelanggan yang sedang login.

- Proteksi Akses Halaman Tambah Motor

File: customers/views.py

```
@customer_required
def motor_add(request):
    """Form tambah motor baru"""
    if request.method == 'POST':
        try:
            # Get form data
            license_plate = request.POST.get('license_plate', '').strip().upper()
            brand = request.POST.get('brand', '').strip()
            model = request.POST.get('model', '').strip()
            year = request.POST.get('year', '').strip()

            print(request.POST)
            # engine_number = request.POST.get('engine_number', '').strip()
            # frame_number = request.POST.get('frame_number', '').strip()
            # notes = request.POST.get('notes', '').strip()

            # Validation
            if not all([license_plate, brand, model, year]):
                messages.error(request, 'Plat nomor, merk, model, tahun, dan warna harus diisi!')
                return render(request, 'customer/dashboard.html', {'post_data': request.POST})

            # Check if license plate already exists
            if Motor.objects.filter(license_plate=license_plate).exists():
                messages.error(request, f'Motor dengan plat nomor {license_plate} sudah terdaftar!')
                return render(request, 'customer/motor_add.html', {'post_data': request.POST})

            # Validate year
            try:
                year_int = int(year)
                if year_int < 1900 or year_int > timezone.now().year + 1:
                    messages.error(request, 'Tahun motor tidak valid!')
                    return render(request, 'customer/motor_add.html', {'post_data': request.POST})
            except ValueError:
                messages.error(request, 'Tahun harus berupa angka!')
                return render(request, 'customer/motor_add.html', {'post_data': request.POST})

            print(request.user)
            # Create motor
            motor = Motor.objects.create(
                owner=request.user,
                license_plate=license_plate,
                brand=brand,
                model=model,
                year=year_int,
```

Penjelasan :

- o Decorator @customer_required memastikan bahwa hanya user dengan role customer yang dapat mengakses fitur tambah motor.
- o Jika user belum login, sistem akan mengarahkan ke halaman login.
- o Jika user bukan customer, sistem menolak akses ke halaman ini.

- Routing URL tambah Motor

File: Customer/urls/.py

```
path('motor/add/', views.motor_add, name='motor_add'),
```

Penjelasan :

- o URL ini digunakan untuk membuka halaman tambah motor dengan `□ar ai /customer/motor/add/`.
- o Request diarahkan ke fungsi motor_add di file customer/views.py.

- Fungsi inti tambah Motor

File: customer/views.py

```
@customer_required
def motor_add(request):
```

Penjelasan :

- o Fungsi ini menangani proses penambahan data motor baru.
- o Fungsi melayani dua kondisi: menampilkan form (GET) dan memproses data form (POST).

- Pengambilan Data Form Input

```
if request.method == 'POST':
    try:
        # Get form data
        license_plate = request.POST.get('license_plate', '').strip().upper()
        brand = request.POST.get('brand', '').strip()
        model = request.POST.get('model', '').strip()
        year = request.POST.get('year', '').strip()

        print(request.POST)
```

Penjelasan :

- o Mengambil data input dari form HTML.
- o strip() digunakan untuk menghilangkan spasi kosong.
- o upper() digunakan agar plat nomor disimpan dalam format huruf besar.

- Validasi kelengkapan data

```
# Validation
if not all([license_plate, brand, model, year]):
    messages.error(request, 'Plat nomor, merk, model, tahun, dan warna harus diisi!')
    return render(request, 'customer/dashboard.html', {'post_data': request.POST})
```

Penjelasan :

- o Memastikan seluruh field wajib telah diisi.
- o Jika ada data kosong, sistem menampilkan pesan error dan tidak melanjutkan proses penyimpanan.

- Validasi Duplikasi Plat Nomor

```
# Check if license plate already exists
if Motor.objects.filter(license_plate=license_plate).exists():
    messages.error(request, f'Motor dengan plat nomor {license_plate} sudah terdaftar!')
    return render(request, 'customer/motor_add.html', {'post_data': request.POST})
```

- Mengecek apakah plat nomor sudah pernah didaftarkan sebelumnya.
- Mencegah satu motor terdaftar lebih dari satu kali.

- Validasi Format Tahun Motor

```
# Validate year
try:
    year_int = int(year)
    if year_int < 1900 or year_int > timezone.now().year + 1:
        messages.error(request, 'Tahun motor tidak valid!')
        return render(request, 'customer/motor_add.html', {'post_data': request.POST})
except ValueError:
    messages.error(request, 'Tahun harus berupa angka!')
    return render(request, 'customer/motor_add.html', {'post_data': request.POST})
print(request.user)
```

Penjelasan :

- Mengubah input tahun menjadi bilangan bulat.
- Memastikan tahun berada pada rentang yang masuk akal.
- Jika format salah, sistem menampilkan pesan kesalahan.

- Penyimpanan Data Motor ke Database

```
# Create motor
motor = Motor.objects.create(
    owner=request.user,
    license_plate=license_plate,
    brand=brand,
    model=model,
    year=year_int,
)

messages.success(request, f'Motor {motor.license_plate} berhasil ditambahkan!')
return redirect('customer:customer_dashboard')
```

Penjelasan :

- Membuat objek motor baru di database.
- Field owner=request.user menghubungkan motor dengan pelanggan yang sedang login.
- Dengan `owner=request.user`, setiap motor hanya dimiliki oleh satu pelanggan

- Notifikasi Berhasil dan Redirect

```
messages.success(request, f'Motor {motor.license_plate} berhasil ditambahkan!')
return redirect('customer:customer_dashboard')
```

- Penanganan error (Exception Handling)

```
except Exception as e:
    messages.error(request, f'Gagal menambahkan motor: {str(e)}')
    return render(request, 'customer/motor_add.html', {'post_data': request.POST})

return render(request, 'customer/motor_add.html')
```

Penjelasan :

- o Menangani kesalahan teknis yang tidak terduga.
- o Menampilkan pesan error tanpa menghentikan aplikasi.

- Tampilan Form Tambah Motor

File: templates/customer/motor_add.html

```
return render(request, 'customer/motor_add.html')
```

4. Halaman customer booking servis

Fitur booking service digunakan oleh pelanggan untuk mendaftarkan antrian servis motor secara online. Melalui fitur ini, pelanggan dapat memilih motor yang telah didaftarkan, memilih jenis layanan servis, serta memasukkan keluhan. Data booking ini selanjutnya akan diproses oleh admin untuk penugasan mekanik dan pengerjaan servis.

- Model Utama Booking Service

File : customer/models.py

```
class ServiceBooking(models.Model):
    """
    Model untuk antrian/booking servis
    """
    STATUS_CHOICES = (
        ('pending', 'Menunggu'),
        ('assigned', 'Ditugaskan ke Mekanik'),
        ('in_progress', 'Sedang Dikerjakan'),
        ('finished', 'Selesai Dikerjakan'),
        ('paid', 'Sudah Dibayar'),
        ('cancelled', 'Dibatalkan'),
    )

    BOOKING_TYPE_CHOICES = (
        ('booking', 'Booking Online'),
        ('walk_in', 'Walk-in'),
    )

    booking_number = models.CharField(max_length=20, unique=True, verbose_name='Nomor Booking')
    motor = models.ForeignKey(Motor, on_delete=models.CASCADE, related_name='bookings')
    customer = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='bookings')
    service_type = models.ForeignKey(ServiceType, on_delete=models.PROTECT, related_name='bookings')
    mechanic = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='assigned_services',
        limit_choices_to={'role': 'mechanic'}
    )

    booking_type = models.CharField(max_length=10, choices=BOOKING_TYPE_CHOICES, default='booking')
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')

    complaint = models.TextField(verbose_name='Keluhan/Permintaan')
    notes = models.TextField(blank=True, null=True, verbose_name='Catatan Tambahan')

    # Timestamps
    booking_date = models.DateTimeField(default=timezone.now, verbose_name='Tanggal Booking')
    assigned_date = models.DateTimeField(null=True, blank=True, verbose_name='Tanggal Ditugaskan')
    start_date = models.DateTimeField(null=True, blank=True, verbose_name='Tanggal Mulai')
    finish_date = models.DateTimeField(null=True, blank=True, verbose_name='Tanggal Selesai')
    payment_date = models.DateTimeField(null=True, blank=True, verbose_name='Tanggal Pembayaran')
```

Penjelasan :

- Class ServiceBooking menyimpan seluruh data antrian servis pelanggan.
- motor menghubungkan booking dengan data motor.
- customer menghubungkan booking dengan akun pelanggan.
- service_type menyimpan jenis layanan yang dipilih.
- status menyimpan status pengerjaan servis.

- Proteksi akses booking servis

```
@customer_required
def booking_create(request):
    """Form buat antrian (Booking/Walk-in)"""
```

Penjelasan :

- Decorator @customer_required memastikan hanya customer yang dapat melakukan booking.

- Jika user belum login atau bukan customer, sistem menolak akses.

- Routing URL Booking Service

File: customer/urls.py

```
path('booking/', views.booking_create, name='booking_create'),
```

Penjelasan :

- URL /customer/booking/ digunakan untuk membuka halaman booking servis.
- Request diteruskan ke fungsi booking_create.

- Menampilkan Data Motor dan Jenis Servis

```
# Get customer's motors
motors = Motor.objects.filter(owner=request.user)

# Get active service types
service_types = ServiceType.objects.filter(is_active=True)
```

Penjelasan :

- Mengambil daftar motor milik customer yang sedang login.
- Mengambil daftar layanan servis yang masih aktif.

- Pengambilan Data Form Booking

```
if request.method == 'POST':
    try:
        # Get form data
        print(request.POST)
        motor_id = request.POST.get('motor')
        first_name = request.POST.get('first_name', '').strip()
        service_type_id = request.POST.get('service')
        complaint = request.POST.get('complaint', '').strip()
        booking_type = request.POST.get('booking_type', 'booking')
```

Penjelasan :

- Mengambil data motor, jenis servis, keluhan, dan tipe booking dari form.

- Validasi Kelengkapan Data

```
# Validation
if not all([motor_id, service_type_id, complaint, booking_type]):
    messages.error(request, 'Motor, jenis servis, dan keluhan harus diisi!')
    return render(request, 'customer/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'post_data': request.POST,
        'booking_type': booking_type,
    })
```

Penjelasan :

Memastikan semua field wajib telah diisi sebelum booking disimpan.

- Pengambilan Data Motor dan Service Type

```
# Get motor and service type
motor = get_object_or_404(Motor, pk=motor_id, owner=request.user)
service_type = get_object_or_404(ServiceType, pk=service_type_id, is_active=True)
```

Penjelasan :

- Mengambil objek motor milik customer.
- Mengambil objek jenis layanan aktif.

- Generate nomor booking

```
# Generate booking number (format: BK-YYYYMMDD-XXXX)
today = timezone.now()
date_str = today.strftime('%Y%m%d')
random_num = random.randint(1000, 9999)
booking_number = f'BK-{date_str}-{random_num}'
```

Penjelasan :

- Membuat nomor booking otomatis dengan format tanggal dan angka acak.
- Menjamin setiap booking memiliki nomor unik.

- Validasi Booking Aktif

```
while ServiceBooking.objects.filter(motor=motor, status__in=['pending', 'in_progress']).exists():
    messages.error(request, f'Motor {motor.license_plate} sudah memiliki booking aktif!')
    return render(request, 'customer/booking_create.html', {
        'motors': motors,
        'service_types': service_types,
        'post_data': request.POST,
        'booking_type': booking_type,
    })
```

Penjelasan :

- Mencegah satu motor memiliki dua booking aktif secara bersamaan.

- Penyimpanan Data Booking

```
booking = ServiceBooking.objects.create(  
    booking_number=booking_number,  
    motor=motor,  
    customer=request.user,  
    service_type=service_type,  
    booking_type='booking',  
    booking_type=booking_type,  
    status='pending',  
    complaint=complaint,  
    booking_date=today,  
)
```

Penjelasan :

- Menyimpan data booking baru ke database.
- Status awal diset sebagai pending (menunggu diproses admin).

- Notifikasi Berhasil dan Redirect

```
messages.success(request, f'Booking berhasil! Nomor booking: {booking.booking_number}')  
return redirect('customer:customer_dashboard')
```

Penjelasan :

- Menampilkan notifikasi booking berhasil.
- Mengarahkan kembali ke dashboard pelanggan.

- Tampilan Form Booking Service

File: templates/customer/booking_create.html

Penjelasan :

- Template menampilkan form pilihan motor, jenis servis, dan input keluhan.
- Data dikirim menggunakan metode POST ke server.

5. Halaman customer Riwayat service

Fitur riwayat servis digunakan oleh pelanggan untuk melihat seluruh data servis yang pernah dilakukan terhadap motor miliknya. Melalui fitur ini, pelanggan dapat memantau status pengerjaan, melihat servis yang telah selesai, serta mengakses detail biaya melalui invoice digital. Fitur ini meningkatkan transparansi dan memberikan catatan histori servis kepada pelanggan.

- Proteksi Akses Halaman Riwayat Servis

File: customer/views.py

```
@customer_required
def service_history(request):
    """Daftar riwayat servis selesai"""
    # Get all bookings for this customer
    bookings = ServiceBooking.objects.filter(
        customer=request.user
    ).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')

    # Filter by status if provided
    status_filter = request.GET.get('status', '')
    if status_filter:
        bookings = bookings.filter(status=status_filter)

    context = {
        'bookings': bookings,
        'status_filter': status_filter,
        'status_choices': ServiceBooking.STATUS_CHOICES,
    }

    return render(request, 'customer/service_history.html', context)
```

```
@customer_required
def service_history(request):
```

Penjelasan :

- Decorator `@customer_required` memastikan hanya user dengan role customer yang dapat mengakses halaman riwayat servis.
- Jika user belum login, sistem akan mengarahkan ke halaman login.
- Jika user bukan customer, sistem menolak akses ke halaman ini.

- Routing URL Riwayat Servis

File: customer/urls.py

```
path('history/', views.service_history, name='service_history'),
```

Penjelasan :

- URL `/customer/history/` digunakan untuk membuka halaman riwayat servis pelanggan. Request diarahkan ke fungsi `service_history` di file `customer/views.py`.

- Fungsi inti Riwayat Servis = file : customer/views.py

```
@customer_required
def service_history(request):
```

Penjelasan :

- Fungsi ini menampilkan seluruh data booking servis milik pelanggan.
- Data ditampilkan berdasarkan akun yang sedang login.

- Pengambilan Data Riwayat Booking

```
# Get all bookings for this customer
bookings = ServiceBooking.objects.filter(
    customer=request.user
).select_related('motor', 'service_type', 'mechanic').order_by('-booking_date')
```

Penjelasan :

- Mengambil seluruh data booking servis milik customer yang sedang login.
- select_related digunakan untuk mengoptimalkan query dengan mengambil data motor, jenis servis, dan mekanik sekaligus.
- Data diurutkan dari booking terbaru ke terlama.

- Fitur Filter Berdasarkan Status

```
# Filter by status if provided
status_filter = request.GET.get('status', '')
if status_filter:
    bookings = bookings.filter(status=status_filter)

context = {
    'bookings': bookings,
    'status_filter': status_filter,
    'status_choices': ServiceBooking.STATUS_CHOICES,
}
```

Penjelasan :

- Sistem membaca parameter status dari URL (query string).

- Jika user memilih filter status, data booking akan difilter sesuai status yang dipilih.
 - Fitur ini memudahkan pelanggan mencari servis tertentu (menunggu, dikerjakan, selesai, dibayar).
- Data Status yang Ditampilkan

```
context = {  
    'bookings': bookings,  
    'status_filter': status_filter,  
    'status_choices': ServiceBooking.STATUS_CHOICES,  
}
```

Penjelasan :

- bookings berisi daftar riwayat servis pelanggan.
- status_filter menyimpan status yang sedang dipilih.
- status_choices digunakan untuk menampilkan daftar pilihan status pada filter di tampilan GUI.

- Menampilkan halaman Riwayat servis pelanggan

```
return render(request, 'customer/service_history.html', context)
```

Penjelasan :

- Sistem menampilkan halaman service_history.html.
 - Seluruh data booking dikirim ke template untuk ditampilkan dalam bentuk tabel atau daftar.
- Tampilan halaman Riwayat servis = File: templates/customer/service_history.html

6. Halaman invoice customer

Fitur invoice digital digunakan untuk menampilkan rincian biaya servis kepada pelanggan setelah proses pengerjaan motor selesai. Melalui fitur ini, pelanggan dapat melihat detail biaya jasa, suku cadang, biaya tambahan, metode pembayaran, serta total pembayaran secara transparan. Invoice digital menjadi bukti transaksi resmi antara bengkel dan pelanggan.

- Model Utama Invoice

File: customer/models.py

```
class Invoice(models.Model):
    """
    Model untuk invoice/nota pembayaran
    """
    PAYMENT_METHOD_CHOICES = (
        ('cash', 'Tunai'),
        ('debit', 'Kartu Debit'),
        ('credit', 'Kartu Kredit'),
        ('transfer', 'Transfer Bank'),
        ('ewallet', 'E-Wallet'),
    )

    invoice_number = models.CharField(max_length=20, unique=True, verbose_name='Nomor Invoice')
    service_booking = models.OneToOneField(ServiceBooking, on_delete=models.CASCADE, related_name='invoice')

    # Biaya
    service_cost = models.DecimalField(max_digits=10, decimal_places=2, verbose_name='Biaya Servis')
    spare_parts_cost = models.DecimalField(max_digits=10, decimal_places=2, default=0, verbose_name='Biaya Suku Cadang')
    additional_cost = models.DecimalField(max_digits=10, decimal_places=2, default=0, verbose_name='Biaya Tambahan')
    total_cost = models.DecimalField(max_digits=10, decimal_places=2, verbose_name='Total Biaya')

    # Pembayaran
    payment_method = models.CharField(max_length=20, choices=PAYMENT_METHOD_CHOICES, verbose_name='Metode Pembayaran')
    paid_amount = models.DecimalField(max_digits=10, decimal_places=2, verbose_name='Jumlah Dibayar')
    change_amount = models.DecimalField(max_digits=10, decimal_places=2, default=0, verbose_name='Kembalian')

    cashier = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name='processed_invoices',
        limit_choices_to={'role': 'cashier'})

    notes = models.TextField(blank=True, null=True, verbose_name='Catatan')
    created_at = models.DateTimeField(auto_now_add=True)
```

Penjelasan :

- Class Invoice merupakan model utama untuk menyimpan data transaksi pembayaran servis.
- service_booking menggunakan relasi OneToOne, artinya satu booking hanya memiliki satu invoice.
- Field biaya digunakan untuk menyimpan rincian perhitungan pembayaran.
- Field cashier menyimpan data kasir yang memproses transaksi.

- Proteksi Akses Halaman Invoice

File: customer/views.py

Penjelasan :

- Decorator `@customer_required` memastikan hanya pelanggan yang dapat mengakses halaman invoice.
- Sistem memverifikasi bahwa invoice hanya dapat diakses oleh pemilik booking terkait.

- Routing URL Invoice

File: customer/urls.py

```
path('invoice/<int:pk>/', views.invoice_detail, name='invoice_detail'),
```

Penjelasan:

- URL `/customer/invoice/<id>/` digunakan untuk membuka halaman detail invoice.
- Parameter `pk` merujuk pada ID booking servis

- Fungsi Inti Invoice Detail

File: customer/views.py

```
@customer_required
def invoice_detail(request, pk):
    """Detail biaya dan nota servis"""
    booking = get_object_or_404(ServiceBooking, pk=pk, customer=request.user)

    # Try to get invoice
    try:
        invoice = Invoice.objects.get(booking=booking)
    except Invoice.DoesNotExist:
        invoice = None

    context = {
        'booking': booking,
        'invoice': invoice,
    }

    return render(request, 'customer/invoice_detail.html', context)
```

Penjelasan:

- Fungsi ini menampilkan detail invoice berdasarkan booking milik customer.

- Pengambilan Data Booking dan Validasi Kepemilikan

```
"""Detail biaya dan nota servis"""
booking = get_object_or_404(ServiceBooking, pk=pk, customer=request.user)
```

Penjelasan:

- Mengambil data booking berdasarkan ID.

- Filter customer=request.user memastikan hanya pemilik booking yang dapat mengakses invoice tersebut.

```
@cashier_or_admin_required
def booking_approve(request, pk):
```

- Pengambilan Data Invoice

```
# Try to get invoice
try:
    invoice = Invoice.objects.get(booking=booking)
except Invoice.DoesNotExist:
    invoice = None
```

Penjelasan:

- Sistem mencoba mengambil data invoice dari database.
- Jika invoice belum dibuat, variabel invoice diisi dengan nilai None.

- Pengiriman data ke template

```
context = {
    'booking': booking,
    'invoice': invoice,
}
```

Penjelasan :

- Data booking dan invoice dikirim ke template untuk ditampilkan.

- Menampilkan Halaman Invoice

```
return render(request, 'customer/invoice_detail.html', context)
```

Penjelasan :

- Sistem menampilkan halaman invoice_detail.html.
- Template menampilkan seluruh rincian biaya dan informasi transaksi

- Tampilan Halaman invoice digital

File: templates/customer/invoice_detail.html

- Perhitungan otomatis biaya

File: customer/models.py

```
def save(self, *args, **kwargs):
    if not self.invoice_number:
        # Generate invoice number: INV-YYYYMMDD-J000X
        from django.utils import timezone
        today = timezone.now().strftime('%Y%m%d')
        last_invoice = Invoice.objects.filter(
            invoice_number__startswith=f'INV-{today}'
        ).order_by('-invoice_number').first()

        if last_invoice:
            last_num = int(last_invoice.invoice_number.split('-')[-1])
            new_num = last_num + 1
        else:
            new_num = 1

        self.invoice_number = f'INV-{today}-{new_num:04d}'

    # calculate total
    self.total_cost = self.service_cost + self.spare_parts_cost + self.additional_cost
    self.change_amount = self.paid_amount - self.total_cost

    super().save(*args, **kwargs)
```

Penjelasan :

- Sistem menghitung total biaya otomatis dari seluruh komponen biaya.
- Sistem menghitung kembalian berdasarkan jumlah pembayaran.
- Proses ini memastikan tidak terjadi kesalahan perhitungan manual.

7. Halaman Dashboard Admin

Fitur dashboard pelanggan pada sistem MotoFix digunakan sebagai halaman utama setelah pengguna dengan peran admin berhasil melakukan login. Dashboard ini berfungsi untuk menampilkan informasi penting terkait data service motor pelanggan, status servis yang sedang berjalan, serta riwayat booking servis pelanggan.

- Routing URL Dashboard Admin

File: admin_hub/urls.py

```
# Dashboard
path('dashboard/', views.admin_dashboard, name='admin_dashboard'),
```

- Proteksi Akses Dashboard Admin

File: admin_hub/views.py

```
# Panel Admin & Manajemen (Integrated Hub)
@admin_required
def admin_dashboard(request):
    """Monitoring seluruh antrian & status bengkel"""
    # Statistik booking
    total_bookings = ServiceBooking.objects.count()
    pending_bookings = ServiceBooking.objects.filter(status='pending').count()
    in_progress_bookings = ServiceBooking.objects.filter(status='in_progress').count()
    finished_bookings = ServiceBooking.objects.filter(status='finished').count()

    context = {
        'total_bookings': total_bookings,
        'pending_bookings': pending_bookings,
        'in_progress_bookings': in_progress_bookings,
        'finished_bookings': finished_bookings,
    }
    return render(request, 'admin_hub/dashboard.html', context)
```

Penjelasan :

- Decorator `@admin_required` memastikan hanya user dengan role admin yang dapat mengakses dashboard ini.
- User non-admin akan ditolak dan diarahkan kembali ke dashboard sesuai rolenya.

- Fungsi Inti Dashboard Admin

File: `admin_hub/views.py`

```
# Panel Admin & Manajemen (Integrated Hub)
@admin_required
def admin_dashboard(request):
    """Monitoring seluruh antrian & status bengkel"""
    # Statistik booking
    total_bookings = ServiceBooking.objects.count()
    pending_bookings = ServiceBooking.objects.filter(status='pending').count()
    in_progress_bookings = ServiceBooking.objects.filter(status='in progress').count()
    finished_bookings = ServiceBooking.objects.filter(status='finished').count()

    context = {
        'total_bookings': total_bookings,
        'pending_bookings': pending_bookings,
        'in_progress_bookings': in_progress_bookings,
        'finished_bookings': finished_bookings,
    }
    return render(request, 'admin_hub/dashboard.html', context)
```

Penjelasan :

- Menghitung total booking servis.
- Menghitung jumlah booking menunggu, sedang dikerjakan, dan selesai.
- Data digunakan sebagai statistik monitoring bengkel.

- Kirim data ke template

```
context = {
    'total_bookings': total_bookings,
    'pending_bookings': pending_bookings,
    'in_progress_bookings': in_progress_bookings,
    'finished_bookings': finished_bookings,
}
```

Penjelasan :

- Mengirim semua data ke halaman HTML
- Agar bisa ditampilkan ke user
- `Total_bookings` menampilkan jumlah booking motor.
- `pending_bookings` menampilkan jumlah booking yang sedang menunggu.
- `in_progress_booking` menampilkan jumlah booking servis yang sedang dilakukan pengerjaan.
- `Finished_booking` menampilkan jumlah booking servis yang sudah selesai dikerjakan.
- data dikirim ke template untuk ditampilkan dalam bentuk statistik dan daftar informasi.

- Menampilkan halaman dashboard

```
return render(request, 'admin_hub/dashboard.html', context)
```

Penjelasan :

- o Menampilkan halaman dashboard.html dengan data yang telah diproses.
- o Template ini berfungsi sebagai tampilan antarmuka (GUI) bagi pelanggan

8. Halaman Admin kelola USER (CRUD USER)

- Routing URL Kelola User

File: admin_hub/urls.py

```
# Manajemen User (CRUD)
path('users/', views.user_list, name='user_list'),
```

- Menampilkan Daftar User

File: admin_hub/views.py

```
# === Manajemen User (CRUD) ===
@admin_required
def user_list(request):
    """List semua user"""
    users = User.objects.all().order_by('-created_at')
    return render(request, 'admin_hub/user_list.html', {'users': users})
```

Penjelasan :

- o Mengambil seluruh data user dari database.
- o Data diurutkan berdasarkan waktu pendaftaran terbaru.

- Menambah user baru

```
# Buat user baru
try:
    user = User.objects.create_user(
        username=username,
        email=email,
        password=password,
        role=role,
        phone=phone,
        address=address,
        first_name=first_name,
        last_name=last_name,
        specialization=specialization
    )
    messages.success(request,
```

Penjelasan :

- o Admin dapat menambahkan user baru (customer, mekanik).
- o Password langsung dienkripsi oleh Django Authentication.

- Update Data User

```
# Update user
try:
    user.username = username
    user.email = email
    user.role = role
    user.phone = phone
    user.address = address
    user.first_name = first_name
    user.last_name = last_name
    user.specialization = specialization
    user.is_active = is_active

    # Update password jika diisi
    new_password = request.POST.get('password')
    if new_password:
        password_confirm = (variable) password_confirm: Any .)
        if new_password != password_confirm:
            messages.error(request, 'Password tidak sama!')
            return render(request, 'admin_hub/user_form.html', {
                'user_obj': user,
                'action': 'update'
            })
        user.set_password(new_password)

    user.save()
```

Penjelasan :

- Admin dapat mengubah data akun user.
- Digunakan untuk koreksi data atau perubahan role.

- Hapus User

```
@admin_required
def user_delete(request, pk):
    """Hapus user"""
    user = get_object_or_404(User, pk=pk)

    if request.method == 'POST':
        username = user.username
        try:
            user.delete()
            messages.success(request, f'User {username} berhasil dihapus!')
            return redirect('admin_hub:user_list')
        except Exception as e:
            messages.error(request, f'Gagal menghapus user: {str(e)}')
            return redirect('admin_hub:user_detail', pk=pk)

    return render(request, 'admin_hub/user_delete.html', {'user_obj': user})
```

Penjelasan :

- Admin dapat menghapus akun user dari sistem.
- Digunakan jika akun tidak aktif atau tidak diperlukan lagi.

9. Halaman Admin Kelola service

Fitur kelola layanan servis digunakan oleh admin untuk mengatur data master jenis layanan yang tersedia di bengkel MotoFix. Melalui fitur ini, admin dapat menambahkan, mengubah, menonaktifkan, dan melihat daftar layanan servis yang dapat dipilih oleh pelanggan pada saat melakukan booking. Fitur ini menjadi dasar utama dalam proses booking dan perhitungan biaya servis.

- Proteksi Akses Halaman Kelola Layanan

File: admin_hub/views.py

```
# === Master Data ===  
@admin_required  
def service_list(request):
```

Penjelasan :

- Decorator `@admin_required` memastikan hanya user dengan role **admin** yang dapat mengakses fitur ini.
- Jika user belum login atau bukan admin, sistem otomatis menolak akses.
- Ini menjaga keamanan master data layanan.

- Routing URL Kelola Layanan

File: admin_hub/urls.py

```
# Master Data  
path('services/', views.service_list, name='service_list'),  
path('services/add/', views.service_add, name='service_add'),  
path('services/<int:pk>/edit/', views.service_edit, name='service_edit'),  
path('services/<int:pk>/delete/', views.service_delete, name='service_delete')
```

Penjelasan :

- `/services/` → menampilkan daftar layanan
- `/services/add/` → membuka form tambah layanan
- `/services/<id>/edit/` → membuka form edit layanan

- Menampilkan Daftar Layanan

Fungsi: `service_list`

```
# === Master Data ===  
@admin_required  
def service_list(request):  
    """List semua jenis layanan (Master Data)"""  
    from customer.models import ServiceType  
    services = ServiceType.objects.all().order_by('name')  
    context = {  
        'services': services,  
    }  
    return render(request, 'admin_hub/service_list.html', context)
```

Penjelasann :

- Mengambil seluruh data layanan dari tabel ServiceType.
- Data diurutkan berdasarkan nama layanan.
- Dikirim ke template untuk ditampilkan dalam tabel daftar layanan.

Fungsi :

- Menampilkan semua layanan aktif dan nonaktif
- Menjadi halaman utama manajemen layanan

- Form Tambah Layanan Baru

```
@admin_required
def service_add(request):
    """Form tambah jenis layanan baru"""
    from customer.models import ServiceType

    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description', '')
        base_price = request.POST.get('base_price')
        estimated_duration = request.POST.get('estimated_duration')

        # Validasi
        if not name or not base_price or not estimated_duration:
            messages.error(request, 'Semua field harus diisi!')
            return render(request, 'admin_hub/service_add.html', {'form_data': request.POST})

        try:
            service = ServiceType.objects.create(
                name=name,
                description=description,
                base_price=float(base_price),
                estimated_duration=int(estimated_duration),
                is_active=True
            )
            messages.success(request, f'Layanan "{name}" berhasil ditambahkan!')
            return redirect('admin_hub:service_list')
        except Exception as e:
            messages.error(request, f'Gagal menambahkan layanan: {str(e)}')
            return render(request, 'admin_hub/service_add.html', {
                'form_data': request.POST
            })

    return render(request, 'admin_hub/service_add.html')
```

Penjelasan :

- Pengambilan data Form

```
if request.method == 'POST':
    name = request.POST.get('name')
    description = request.POST.get('description', '')
    base_price = request.POST.get('base_price')
    estimated_duration = request.POST.get('estimated_duration')
```

Penjelasan: Mengambil input nama layanan, deskripsi, harga dasar, dan estimasi durasi.

- Validasi Input Layanan

```
# Validasi
if not name or not base_price or not estimated_duration:
    messages.error(request, 'Semua field harus diisi!')
    return render(request, 'admin_hub/service_add.html', {'form_data': request.POST})
```

Penjelasan:

- Memastikan field wajib tidak kosong.
- Jika ada data kosong, proses dibatalkan dan admin diberi notifikasi error.

- Penyimpanan Data Layanan ke Database

```
try:
    service = ServiceType.objects.create(
        name=name,
        description=description,
        base_price=float(base_price),
        estimated_duration=int(estimated_duration),
        is_active=True
    )
```

Penjelasan :

- Membuat record layanan baru di tabel service_types.
- Harga dikonversi ke tipe float.
- Durasi dikonversi ke integer.
- is_active=True menandakan layanan langsung aktif.

- Notifikasi Berhasil dan Redirect

```
messages.success(request, f'Layanan "{name}" berhasil ditambahkan!')
return redirect('admin_hub:service_list')
```

Penjelasan :

- Menampilkan pesan sukses.
- Admin diarahkan kembali ke halaman daftar layanan.

- Edit Data Layanan

Fungsi: service_edit

```
@admin_required
def service_edit(request, pk):
    """Edit jenis layanan"""
    from customer.models import ServiceType
    service = get_object_or_404(ServiceType, pk=pk)

    if request.method == 'POST':
        name = request.POST.get('name')
        description = request.POST.get('description', '')
        base_price = request.POST.get('base_price')
        estimated_duration = request.POST.get('estimated_duration')
        is_active = request.POST.get('is_active') == 'on'

        # Validasi
        if not name or not base_price or not estimated_duration:
            messages.error(request, 'Semua field harus diisi!')
            return render(request, 'admin_hub/service_edit.html', {
                'service': service,
            })

        try:
            service.name = name
            service.description = description
            service.base_price = float(base_price)
            service.estimated_duration = int(estimated_duration)
            service.is_active = is_active
            service.save()
            messages.success(request, f'Layanan "{name}" berhasil diupdate!')
            return redirect('admin_hub:service_list')
        except Exception as e:
            messages.error(request, f'Gagal mengupdate layanan: {str(e)}')
            return render(request, 'admin_hub/service_edit.html', {
                'service': service,
            })

    return render(request, 'admin_hub/service_edit.html', {
        'service': service,
    })
```

- Mengambil Data Layanan Berdasarkan ID

```
"""Edit jenis layanan"""
from customer.models import ServiceType
service = get_object_or_404(ServiceType, pk=pk)
```

Penjelasan :

- o Mengambil satu data layanan berdasarkan primary key.
- o Jika data tidak ditemukan, sistem menampilkan halaman 404.

- Update Data Layanan

```
try:
    service.name = name
    service.description = description
    service.base_price = float(base_price)
    service.estimated_duration = int(estimated_duration)
    service.is_active = is_active
    service.save()
    messages.success(request, f'Layanan "{name}" berhasil diupdate!')
    return redirect('admin_hub:service_list')
except Exception as e:
    messages.error(request, f'Gagal mengupdate layanan: {str(e)}')
    return render(request, 'admin_hub/service_edit.html', {
        'service': service,
    })
```

Penjelasan :

- Memperbarui data layanan sesuai input admin.
- Field is_active digunakan untuk mengaktifkan atau menonaktifkan layanan.
- Perubahan langsung disimpan ke database.

- Penanganan Error (Exception Handling)

```
except Exception as e:
    messages.error(request, f'Gagal mengupdate layanan: {str(e)}')
    return render(request, 'admin_hub/service_edit.html', {
        'service': service,
    })
```

Penjelasan :

- Menangkap kesalahan teknis saat penyimpanan data.
- Menampilkan pesan error tanpa menghentikan sistem.

- Tampilan Antarmuka Kelola Layanan

File:

- templates/admin_hub/service_list.html
- templates/admin_hub/service_add.html
- templates/admin_hub/service_edit.htm

10. Halaman admin Kelola mekanik

Halaman kelola mekanik digunakan oleh admin untuk melihat daftar seluruh mekanik yang terdaftar di sistem. Data mekanik ini nantinya digunakan pada proses penugasan mekanik ke booking servis pelanggan. Dengan fitur ini, admin dapat memastikan hanya user dengan peran mekanik yang dapat ditugaskan menangani servis.

- Model User sebagai Data Mekanik

File: admin_hub/views.py

```
@admin_required
def mechanic_list(request):
    """List data mekanik"""
    mechanics = User.objects.filter(role='mechanic')
    return render(request, 'admin_hub/mechanic_list.html', {'mechanics': mechanics})
```

Penjelasan :

- o Decorator `@admin_required` memastikan hanya user dengan role admin yang dapat mengakses fitur ini.
- o Jika user belum login atau bukan admin, sistem otomatis menolak akses.
- o Ini menjaga keamanan master data layanan.

- Routing URL Kelola Layanan

File: admin_hub/urls.py

```
path('mechanics/', views.mechanic_list, name='mechanic_list'),
```

- Menampilkan data mekanik

```
@admin_required
def mechanic_list(request):
    """List data mekanik"""
    mechanics = User.objects.filter(role='mechanic')
    return render(request, 'admin_hub/mechanic_list.html', {'mechanics': mechanics})
```

Penjelasan :

- o Mengambil seluruh user dengan role mekanik.
- o Digunakan sebagai dasar penugasan mekanik pada booking servis.

11. Halaman Invoice Admin

Fitur Invoice Admin digunakan oleh admin/kasir untuk memproses pembayaran servis pelanggan dan membuat invoice digital resmi. Melalui fitur ini, admin dapat memasukkan rincian biaya jasa, suku cadang, biaya tambahan, metode pembayaran, serta menghasilkan invoice sebagai bukti transaksi.

- Routing Halaman Service Admin

File: admin_hub/views.py

```
@cashier_or_admin_required
def invoice_detail(request, pk):
    """Detail invoice untuk admin/kasir"""
    from customer.models import Invoice, SparePart, AdditionalService

    booking = get_object_or_404(ServiceBooking, pk=pk)

    # Get invoice
    try:
        invoice = Invoice.objects.get(service_booking=booking)
    except Invoice.DoesNotExist:
        messages.error(request, 'Invoice belum dibuat untuk booking ini!')
        return redirect('admin_hub:booking_detail', pk=pk)

    # Get spare parts and additional services
    spare_parts = SparePart.objects.filter(service_booking=booking)
    additional_services = AdditionalService.objects.filter(service_booking=booking)

    context = {
        'booking': booking,
        'invoice': invoice,
        'spare_parts': spare_parts,
        'additional_services': additional_services,
    }

    return render(request, 'admin_hub/invoice_detail.html', context)
```

Penjelasan :

- Fungsi ini menampilkan halaman form pembayaran berdasarkan booking yang dipilih.
- Parameter pk digunakan untuk mengambil data booking terkait.

- Model Utama Invoice

File : admin_hub/views.py

```
@cashier_or_admin_required
def create_invoice(request, pk):
    """Buat invoice untuk booking yang sudah selesai"""
    from customer.models import Invoice, SparePart, AdditionalService
    from decimal import Decimal

    booking = get_object_or_404(ServiceBooking, pk=pk)

    # Cek apakah booking sudah selesai atau paid
    if booking.status not in ['finished', 'paid']:
        messages.error(request, 'Invoice hanya bisa dibuat untuk booking yang sudah selesai!')
        return redirect('admin_hub:booking_detail', pk=pk)

    # Cek apakah invoice sudah ada
    try:
        existing_invoice = Invoice.objects.get(service_booking=booking)
        messages.warning(request, 'Invoice sudah dibuat sebelumnya!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Invoice.DoesNotExist:
        pass

    if request.method == 'POST':
        payment_method = request.POST.get('payment_method', 'cash')

        try:
            # Hitung biaya layanan dasar
            service_cost = Decimal(str(booking.service_type.base_price))

            # Hitung total biaya suku cadang
            spare_parts = SparePart.objects.filter(service_booking=booking)
            spare_parts_cost = sum(Decimal(str(sp.price * sp.quantity)) for sp in spare_parts)

            # Hitung total biaya jasa tambahan
            additional_services = AdditionalService.objects.filter(service_booking=booking)
            additional_cost = sum(Decimal(str(ads.price)) for ads in additional_services)

            # Total biaya
            total_cost = service_cost + spare_parts_cost + additional_cost

            # Buat invoice
            invoice = Invoice.objects.create(
                service_booking=booking,
                invoice_number=f"INV-{booking.booking_number}",

def create_invoice(request, pk):
    # Hitung total biaya suku cadang
    spare_parts = SparePart.objects.filter(service_booking=booking)
    spare_parts_cost = sum(Decimal(str(sp.price * sp.quantity)) for sp in spare_parts)

    # Hitung total biaya jasa tambahan
    additional_services = AdditionalService.objects.filter(service_booking=booking)
    additional_cost = sum(Decimal(str(ads.price)) for ads in additional_services)

    # Total biaya
    total_cost = service_cost + spare_parts_cost + additional_cost

    # Buat invoice
    invoice = Invoice.objects.create(
        service_booking=booking,
        invoice_number=f"INV-{booking.booking_number}",
        service_cost=service_cost,
        spare_parts_cost=spare_parts_cost,
        additional_cost=additional_cost,
        total_cost=total_cost,
        payment_method=payment_method,
        paid_amount=total_cost,
        change_amount=Decimal('0'),
        cashier=request.user
    )

    # Update status booking menjadi paid jika belum
    if booking.status != 'finished':
        booking.status = 'paid'
        booking.save()

    messages.success(request, f'Invoice {invoice.invoice_number} berhasil dibuat! Total: Rp {total_cost:,0f}')
    return redirect('admin_hub:booking_detail', pk=pk)

except Exception as e:
    messages.error(request, f'Gagal membuat invoice: {str(e)}')
    return redirect('admin_hub:booking_detail', pk=pk)

return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- @cashier_or_admin_required Hanya admin / kasir yang boleh membuat invoice. Customer & mekanik TIDAK BISA akses
- pk ID dari **booking servis** yang akan dibayar

- Ambil data booking

```
booking = get_object_or_404(ServiceBooking, pk=pk)
```

Penjelasan :

- Mengambil data booking dari database berdasarkan ID
- Kalau ID salah maka otomatis error 404

Ini booking yang mau dibuatkan invoice

- Cek status booking apakah sudah ada

```
# Cek apakah booking sudah selesai atau paid
if booking.status not in ['finished', 'paid']:
    messages.error(request, 'Invoice hanya bisa dibuat untuk booking yang sudah selesai!')
    return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- Invoice HANYA BOLEH dibuat jika:
 - Servis sudah selesai
 - Atau sudah paid
 - Tidak boleh buat invoice jika:
 - Masih pending
 - Masih dikerjakan
- Cek apakah invoice udah ada

```
# Cek apakah invoice sudah ada
try:
    existing_invoice = Invoice.objects.get(service_booking=booking)
    messages.warning(request, 'Invoice sudah dibuat sebelumnya!')
    return redirect('admin_hub:booking_detail', pk=pk)
except Invoice.DoesNotExist:
    pass
```

Penjelasan :

- Mengecek apakah booking ini sudah pernah dibuatkan invoice
 - Kalau sudah:
 - Sistem menolak
 - Tidak boleh dobel invoice
- Cek method post (tombol disubmit)

```
if request.method == 'POST':
    payment_method = request.POST.get('payment_method', 'cash')
```

Penjelasan :

- Kode ini dijalankan saat:
Admin menekan tombol **“Proses Pembayaran”**
- payment_method diambil dari form:
 - cash

- debit
- transfer
- ewallet

- Hitung biaya dasar

```
try:
    # Hitung biaya layanan dasar
    service_cost = Decimal(str(booking.service_type.base_price))
```

Penjelasan :

- Mengambil **harga layanan utama** dari tabel ServiceType

Contoh:

- Ganti oli = 35.000
- Servis besar = 150.000
- Ini biaya dasar servis

- Hitung total biaya suku cadang

```
# Hitung total biaya suku cadang
spare_parts = SparePart.objects.filter(service_booking=booking)
spare_parts_cost = sum(Decimal(str(sp.price * sp.quantity)) for sp in spare_parts)
```

Penjelasan :

- Ambil semua suku cadang yang dipakai di servis ini
- Hitung: harga x jumlah

- Hitung biaya tambahan jasa

```
# Hitung total biaya jasa tambahan
additional_services = AdditionalService.objects.filter(service_booking=booking)
additional_cost = sum(Decimal(str(ads.price)) for ads in additional_services)
```

Penjelasan :

- Digunakan jika ada:
 - Cuci motor
 - Setel rem
 - Tambah tune up
- Total biaya jasa tambahan dijumlahkan

- Hitung total keseluruhan

```
# Total biaya
total_cost = service_cost + spare_parts_cost + additional_cost
```

Penjelasan : Total = Biaya Servis + Biaya Spare Part + Biaya Tambahan

- Membuat data invoice ke database

```
# Buat invoice
invoice = Invoice.objects.create(
    service_booking=booking,
    invoice_number=f"INV-{booking.booking_number}",
    service_cost=service_cost,
    spare_parts_cost=spare_parts_cost,
    additional_cost=additional_cost,
    total_cost=total_cost,
    payment_method=payment_method,
    paid_amount=total_cost,
    change_amount=Decimal('0'),
    cashier=request.user
)
```

Penjelasan :

- Membuat **1 record invoice baru**
 - Menyimpan:
 - Nomor invoice
 - Semua biaya
 - Metode pembayaran
 - Kasir yang memproses
 - Relasi penting:
 - service_booking=booking
Invoice terhubung langsung ke booking
- Ubah status booking jadi paid

```
# Update status booking menjadi paid jika belum
if booking.status == 'finished':
    booking.status = 'paid'
    booking.save()
```

Penjelasan :

- Jika servis sudah selesai
- Setelah bayar → status diubah menjadi **paid**
- Ini menandakan:
 - Transaksi selesai
 - Booking ditutup
 - Tidak bisa diubah lagi

- Tampilkan status berhasil

```
messages.success(request, f'Invoice {invoice.invoice_number} berhasil dibuat! Total: Rp {total_cost:,.0f}')
return redirect('admin_hub:booking_detail', pk=pk)
```

12. Halaman Kelola Admin Booking Pelanggan

Fitur kelola data booking servis digunakan oleh admin untuk mengatur seluruh proses pemesanan servis motor, mulai dari persetujuan booking, penugasan mekanik, pembatalan booking, hingga pemantauan status pengerjaan. Fitur ini merupakan inti dari sistem operasional bengkel MotoFix karena menghubungkan pelanggan, mekanik, dan proses pembayaran.

- Routing URL Kelola Booking

File: admin_hub/urls.py

```
# Manajemen Booking Service
path('bookings/', views.booking_list, name='booking_list'),
path('bookings/create/', views.booking_create, name='booking_create'),
path('bookings/<int:pk>/', views.booking_detail, name='booking_detail'),
path('bookings/<int:pk>/approve/', views.booking_approve, name='booking_approve'),
path('bookings/<int:pk>/assign-mechanic/', views.booking_assign_mechanic, name='booking_assign_mechanic'),
path('bookings/<int:pk>/start/', views.booking_start, name='booking_start'),
path('bookings/<int:pk>/finish/', views.booking_finish, name='booking_finish'),
path('bookings/<int:pk>/cancel/', views.booking_cancel, name='booking_cancel'),
path('bookings/<int:pk>/update-status/', views.booking_update_status, name='booking_update_status'),
```

Penjelasan :

- o /bookings/ untuk menampilkan daftar seluruh booking
- o /bookings/<id>/ untuk menampilkan detail booking
- o /approve/ untuk menyetujui booking
- o /assign-mechanic/ untuk menugaskan mekanik
- o /cancel/ untuk membatalkan booking
- o /start/ untuk memulai pengerjaan
- o Finish/ untuk menyelesaikan pengerjaan

- Proteksi Akses Halaman Booking

```
# === Manajemen Booking Service ===
@staff_required
def booking_list(request):
    ....

@casher_or_admin_required
def booking_approve(request, pk):
```

Penjelasan :

- o @staff_required memastikan hanya staff dan admin yang dapat melihat daftar booking
- o @casher_or_admin_required memastikan hanya kasir dan admin yang dapat memproses booking
- o Proteksi ini mencegah pelanggan mengakses fitur internal bengkel

- Menampilkan Daftar Booking Servis
Fungsi: booking_list

```
bookings = ServiceBooking.objects.select_related(
    'customer', 'motor', 'service_type', 'mechanic'
).order_by('-booking_date')
```

Penjelasan :

- o Mengambil seluruh data booking servis dari database
 - o select_related digunakan untuk mempercepat akses relasi
 - o Data diurutkan dari booking terbaru
 - o Data yang ditampilkan :
 - Nomor booking
 - Nama pelanggan
 - Data motor
 - Jenis layanan
 - Status booking
- Menampilkan Detail Booking & Daftar Mekanik
Fungsi: booking_detail

```
booking = get_object_or_404(ServiceBooking.objects.select_related(
    'customer', 'motor', 'service_type', 'mechanic'
).prefetch_related('spare_parts', 'additional_services'), pk=pk)

# include role case-insensitive to avoid missing mechanics due to unexpected role values
mechanics = User.objects.filter(role__iexact='mechanic').order_by('first_name', 'username')
```

Penjelasan :

- o Mengambil satu data booking berdasarkan ID
 - o Mengambil daftar seluruh mekanik
 - o Data ini digunakan admin untuk memilih mekanik yang akan ditugaskan
- Approve Booking
Fungsi: booking_approve

```
if booking.status != 'pending':
    messages.error(request, 'Booking hanya bisa di-approve jika status Menunggu!')
    return redirect('admin_hub:booking_detail', pk=pk)

try:
    booking.status = 'assigned'
    booking.assigned_date = timezone.now()
    booking.save()
    messages.success(request, f'Booking {booking.booking_number} berhasil di-approve!')
    return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- o Booking hanya bisa di-approve jika status masih pending
- o Status booking berubah menjadi assigned
- o Waktu persetujuan dicatat pada assigned_date
- o Fungsi
 - Menandai booking telah disetujui bengkel

- Booking siap ditugaskan ke mekanik
- Menugaskan mekanik ke booking
Fungsi : booking assign mechanic

```

def booking_assign_mechanic(request, pk):
    """Assign mekanik ke booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if not mechanic_id:
        messages.error(request, 'Pilih mekanik terlebih dahulu!')
        return redirect('admin_hub:booking_detail', pk=pk)

    try:
        # Try to get the user by id first. Avoid failing if role string differs.
        mechanic = User.objects.get(id=mechanic_id)
        if not mechanic.is_mechanic:
            messages.error(request, 'User yang dipilih bukan mekanik. Periksa peran user.')
            return redirect('admin_hub:booking_detail', pk=pk)
        booking.mechanic = mechanic
        booking.status = 'assigned'
        booking.assigned_date = timezone.now()
        booking.save()
        messages.success(request, f'Mekanik {mechanic.get_full_name()} berhasil ditugaskan!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except User.DoesNotExist:
        messages.error(request, 'Mekanik tidak ditemukan!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal assign mekanik: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)

    return redirect('admin_hub:booking_detail', pk=pk)

```

Penjelasan :

- Admin memilih mekanik berdasarkan ID
- Sistem memvalidasi bahwa user benar-benar mekanik
- Mekanik disimpan pada field booking.mechanic
- Status booking menjadi assigned
- Fungsi
 - Menghubungkan booking dengan mekanik
 - Menentukan siapa yang akan mengerjakan servis

- Membatalkan Booking Servis

Fungsi : booking_cancel

```
@cashier_or_admin_required
def booking_cancel(request, pk):
    """Batalan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status in ['finished', 'paid', 'cancelled']:
        messages.error(request, f'Booking dengan status {booking.get_status_display()} tidak bisa dibatalkan!')
        return redirect('admin_hub:booking_detail', pk=pk)

    if request.method == 'POST':
        try:
            booking.status = 'cancelled'
            booking.save()
            messages.success(request, f'Booking {booking.booking_number} berhasil dibatalkan!')
            return redirect('admin_hub:booking_list')
        except Exception as e:
            messages.error(request, f'Gagal membatalkan booking: {str(e)}')
            return redirect('admin_hub:booking_detail', pk=pk)

    return render(request, 'admin_hub/booking_cancel.html', {'booking': booking})
```

```
if booking.status in ['finished', 'paid', 'cancelled']:
    messages.error(request, f'Booking dengan status {booking.get_status_display()} tidak bisa dibatalkan!')
    return redirect('admin_hub:booking_detail', pk=pk)

if request.method == 'POST':
    try:
        booking.status = 'cancelled'
        booking.save()
        messages.success(request, f'Booking {booking.booking_number} berhasil dibatalkan!')
        return redirect('admin_hub:booking_list')
    except Exception as e:
        messages.error(request, f'Gagal membatalkan booking: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)

return render(request, 'admin_hub/booking_cancel.html', {'booking': booking})
```

Penjelasan :

- Booking yang sudah selesai atau dibayar tidak dapat dibatalkan
- Status booking diubah menjadi cancelled
- Booking tidak akan diproses lebih lanjut

- Booking Start

Fungsi : booking_start

```
@cashier_or_admin_required
def booking_start(request, pk):
    """Mulai pengerjaan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status != 'assigned':
        messages.error(request, 'Booking hanya bisa dimulai jika sudah ditugaskan!')
        return redirect('admin_hub:booking_detail', pk=pk)

    try:
        booking.status = 'in_progress'
        booking.start_date = timezone.now()
        booking.save()
        messages.success(request, f'Booking {booking.booking_number} mulai dikerjakan!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal memulai booking: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- Sistem mengambil data booking berdasarkan ID
- Validasi dilakukan:
 - Servis hanya boleh dimulai jika status booking = assigned
- Jika valid:
 - Status booking diubah menjadi in_progress
 - Waktu mulai servis dicatat pada field start_date
 - Perubahan disimpan ke database
- Fungsi :
 - Menandai bahwa mekanik mulai mengerjakan motor
 - Digunakan untuk monitoring antrian bengkel

- Booking finish

Fungsi : booking_finish

```
@cashier_or_admin_required
def booking_finish(request, pk):
    """Selesaikan booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if booking.status != 'in_progress':
        messages.error(request, 'Booking hanya bisa diselesaikan jika sedang dikerjakan!')
        return redirect('admin_hub:booking_detail', pk=pk)

    try:
        booking.status = 'finished'
        booking.finish_date = timezone.now()
        booking.save()
        messages.success(request, f'Booking {booking.booking_number} telah selesai dikerjakan!')
        return redirect('admin_hub:booking_detail', pk=pk)
    except Exception as e:
        messages.error(request, f'Gagal menyelesaikan booking: {str(e)}')
        return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- Sistem hanya mengizinkan penyelesaian servis jika status booking = in_progress
- Jika valid:
 - Status booking diubah menjadi finished

- Waktu selesai servis dicatat pada field finish_date
 - Data disimpan ke database
 - Fungsi:
 - Menandai bahwa pengerjaan mekanik telah selesai
 - Menjadi syarat sebelum invoice dapat dibuat
- Booking update
- Fungsi : booking_update

```
@cashier_or_admin_required
def booking_update_status(request, pk):
    """Update status booking"""
    booking = get_object_or_404(ServiceBooking, pk=pk)

    if request.method == 'POST':
        new_status = request.POST.get('status')

        # Validasi transisi
        valid_transitions = {
            'pending': ['assigned', 'cancelled'],
            'assigned': ['in_progress', 'cancelled'],
            'in_progress': ['finished', 'cancelled'],
            'finished': ['paid'],
            'paid': [],
            'cancelled': [],
        }

        if new_status not in valid_transitions.get(booking.status, []):
            messages.error(request, f'Transisi status dari {booking.get_status_display()} ke {new_status} tidak diizinkan!')
            return redirect('admin_hub:booking_detail', pk=pk)

        try:
            booking.status = new_status

            # Update timestamp sesuai status
            if new_status == 'assigned' and not booking.assigned_date:
                booking.assigned_date = timezone.now()
            elif new_status == 'in_progress' and not booking.start_date:
                booking.start_date = timezone.now()
            elif new_status == 'finished' and not booking.finish_date:
                booking.finish_date = timezone.now()
            elif new_status == 'paid' and not booking.payment_date:
                booking.payment_date = timezone.now()

            booking.save()
            messages.success(request, f'Status booking berhasil diubah menjadi {booking.get_status_display()}!')
            return redirect('admin_hub:booking_detail', pk=pk)
        except Exception as e:
            messages.error(request, f'Gagal update status: {str(e)}')
            return redirect('admin_hub:booking_detail', pk=pk)

    return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- Pengambilan Data Booking

```
booking = get_object_or_404(ServiceBooking, pk=pk)
```

Penjelasan :

- Mengambil satu data booking berdasarkan ID
- Jika booking tidak ditemukan, sistem menampilkan halaman error 404

- Pengambilan Status Baru dari Form

```
if request.method == 'POST':
    new_status = request.POST.get('status')
```

Penjelasan :

- Sistem mengambil nilai status baru dari input form

- Status baru dikirim melalui metode POST
- Validasi Transisi Status Booking

```
# Validasi status transition
valid_transitions = {
    'pending': ['assigned', 'cancelled'],
    'assigned': ['in_progress', 'cancelled'],
    'in_progress': ['finished', 'cancelled'],
    'finished': ['paid'],
    'paid': [],
    'cancelled': [],
}
```

Penjelasan :

- Dictionary ini mendefinisikan alur status yang diizinkan
- Mencegah perubahan status yang tidak logis, misalnya:
 - pending langsung ke paid
 - finished kembali ke assigned
- Validasi perubahan status

```
if new_status not in valid_transitions.get(booking.status, []):
    messages.error(request, f'Transisi status dari {booking.get_status_display()} ke {new_status} tidak diizinkan!')
    return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

- Sistem mengecek apakah perubahan status diperbolehkan.
- Jika tidak sesuai alur:
 - Proses dibatalkan
 - Admin mendapat pesan error
- Update status dan timestamp otomatis

```
try:
    booking.status = new_status

    # Update timestamp sesuai status
    if new_status == 'assigned' and not booking.assigned_date:
        booking.assigned_date = timezone.now()
    elif new_status == 'in_progress' and not booking.start_date:
        booking.start_date = timezone.now()
    elif new_status == 'finished' and not booking.finish_date:
        booking.finish_date = timezone.now()
    elif new_status == 'paid' and not booking.payment_date:
        booking.payment_date = timezone.now()

    booking.save()
```

Penjelasan:

- Status booking diperbarui sesuai pilihan admin
- Sistem otomatis mencatat waktu perubahan status:

- assigned_date → waktu ditugaskan
 - start_date → waktu mulai servis
 - finish_date → waktu selesai servis
 - payment_date → waktu pembayaran
- Penyimpanan perubahan ke database

`booking.save()`

Penjelasan :

 - Seluruh perubahan status dan waktu disimpan permanen ke database
 - Data ini digunakan untuk laporan, monitoring, dan histori servis
 - Notifikasi dan redirect


```
messages.success(request, f'Status booking berhasil diubah menjadi {booking.get_status_display()}!')
return redirect('admin_hub:booking_detail', pk=pk)
```

Penjelasan :

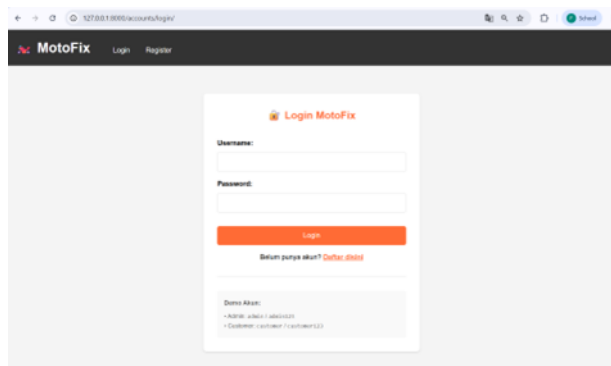
 - Menampilkan pesan sukses kepada admin
 - Admin diarahkan kembali ke halaman detail booking

g. Penjelasan screenshot tampilan yang dihasilkan aplikasi

1. User Pelanggan

- Form Login

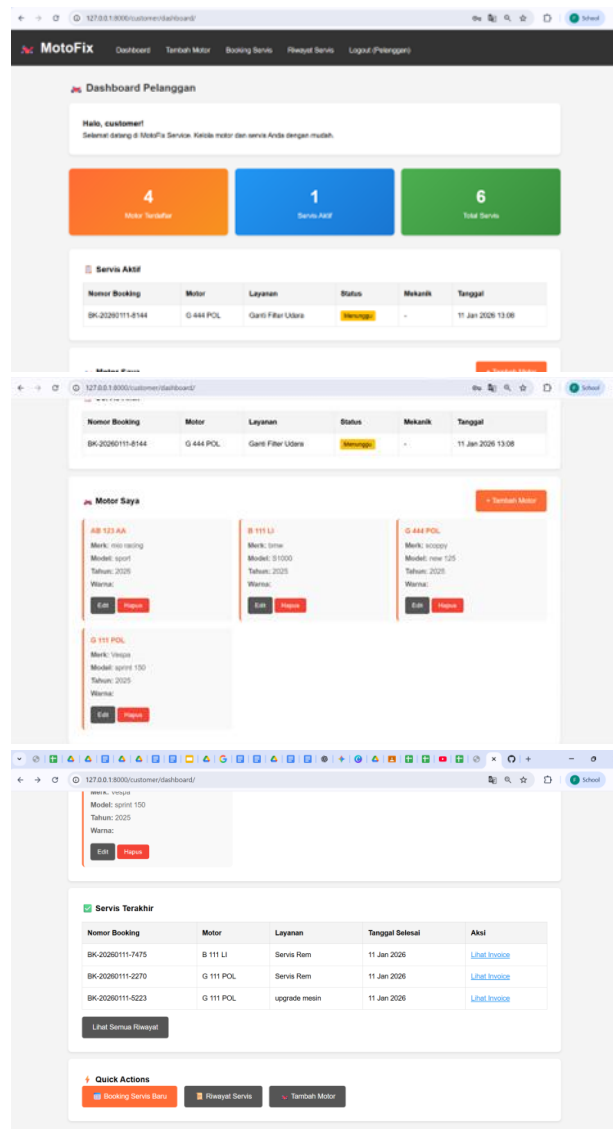
Menampilkan form login yang digunakan oleh pelanggan dan admin untuk mengakses sistem. Pengguna harus memasukkan username dan password yang valid.



Gambar Halaman Login Pelanggan

- Dashboard Pelanggan

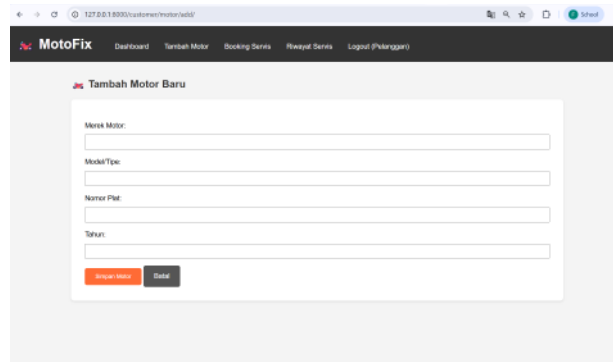
Dashboard pelanggan menampilkan ringkasan informasi berupa jumlah motor terdaftar, servis aktif, dan total riwayat servis. Selain itu, pelanggan dapat melihat status pengerjaan motor secara real-time.



Gambar Halaman Dashboard Pelanggan

- Halaman Tambah Motor Pelanggan

Digunakan pelanggan untuk menambahkan data motor yang akan diservis, meliputi merk, tipe, dan nomor plat kendaraan, tahun produksi kendaraan.

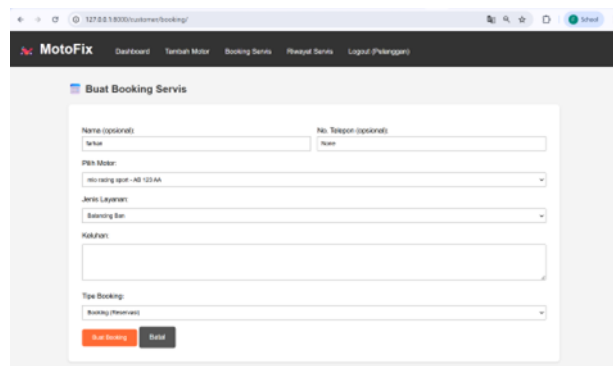


The screenshot shows a web browser window with the URL '127.0.0.1:8000/customer/motor/add/'. The page title is 'MotoFix' and the navigation bar includes 'Dashboard', 'Tambah Motor', 'Booking Servis', 'Riwayat Servis', and 'Logout (Pelanggan)'. The main heading is 'Tambah Motor Baru'. The form contains four input fields: 'Merek Motor', 'Model/Tipe', 'Nomor Plat', and 'Tahun'. At the bottom of the form are two buttons: 'Simpan Motor' (orange) and 'Batal' (grey).

Gambar Halaman Tambah Motor Pelanggan

- Halaman booking servis motor pelanggan

Digunakan pelanggan untuk melakukan booking servis motor pelanggan bisa memilih antara (booking dan walk-in).



The screenshot shows a web browser window with the URL '127.0.0.1:8000/customer/booking/'. The page title is 'MotoFix' and the navigation bar includes 'Dashboard', 'Tambah Motor', 'Booking Servis', 'Riwayat Servis', and 'Logout (Pelanggan)'. The main heading is 'Buat Booking Servis'. The form contains several input fields: 'Name (optional)' and 'No. Telepon (optional)' (both with 'Name' placeholder text), 'PIN Motor' (with a dropdown menu showing 'No rekening bank - AD 123 AA'), 'Jenis Layanan' (with a dropdown menu showing 'Servis Ban'), 'Keterangan' (a text area), and 'Type Booking' (a dropdown menu showing 'Booking (Reservasi)'). At the bottom of the form are two buttons: 'Buat Booking' (orange) and 'Batal' (grey).

Gambar halaman booking servis motor pelanggan

- Halaman Riwayat servis

Digunakan pelanggan untuk menampilkan riwayat data motor yang telah diservis.

Nomor Booking	Tanggal	Motor	Layanan	Status	Aksi
BK-20260114-0905	14 Jan 2026 11:04	AB 330204 CD Honda Beat	Cuci Motor	Selesai	Ulangi Servis
BK-20260114-0902	14 Jan 2026 11:45	AB 330204 CD Honda Beat	Ganti Ban	Selesai	Ulangi Servis
BK-20260114-0901	14 Jan 2026 11:46	AB 330204 CD Honda Beat	oil	Selesai	Ulangi Servis
BK-20251230-1631	30 Des 2025 16:19	AB 330204 CD Honda Beat	Ganti Kampas Rem	Selesai	Ulangi Servis
BK-20251230-1603	30 Des 2025 16:19	AB 330204 CD Honda Beat	Cuci Motor	Selesai	Ulangi Servis
BK-20251230-0509	30 Des 2025 15:59	AB 330204 CD Honda Beat	Cuci Motor	Selesai	Ulangi Servis

Gambar halaman Riwayat servis motor pelanggan

- Halaman customer cetak invoice

Digunakan pelanggan untuk menampilkan/mencetak invoice hasil total biaya service yang harus dibayar

1/19/26, 8:03 PM Invoice Detail - MotoFix

Invoice Servis #INV-BK-20260111-7475

Informasi Servis
 Nomor Booking: BK-20260111-7475
 Tanggal Booking: 11 Januari 2026 22:02
 Tanggal Selesai: 11 Januari 2026 22:05
 Motor: bmw S1000 - B 111 LI
 Mekanik: Bapak Roy
 Layanan: Servis Rem
 Status: Sudah Dibayar

Item	Jumlah
Biaya Jasa Servis (Servis Rem)	Rp 75000
TOTAL:	Rp 75000
Dibayar (Tunai)	Rp 75000
Kembalian	Rp 0

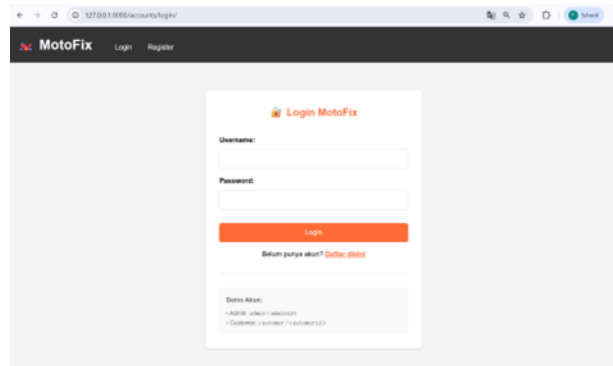
Kasir:
 Tanggal Invoice: 19 Januari 2026 14:24

Gambar halaman pelanggan cetak invoice

2. User Admin

- Halaman Login

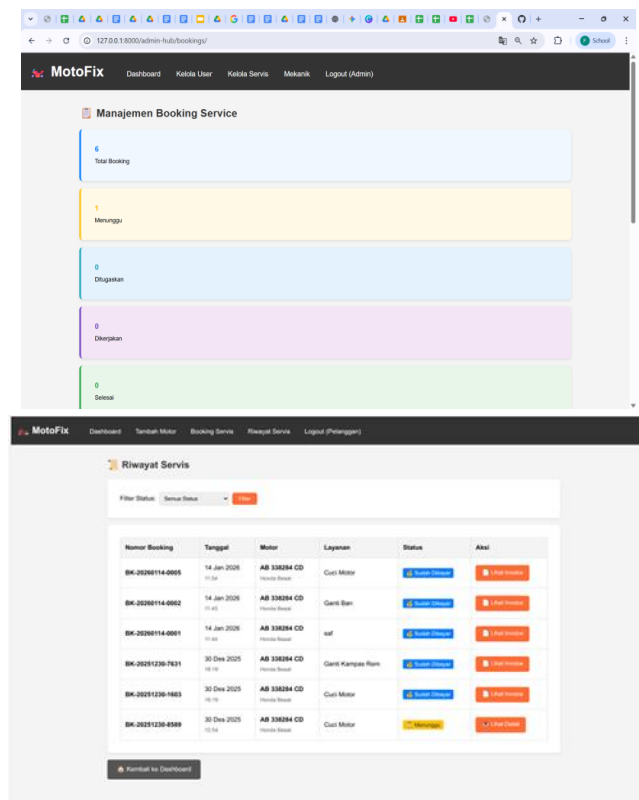
Menampilkan form login yang digunakan oleh admin dan untuk mengakses sistem. Admin harus memasukkan username dan password yang valid.



Gambar halaman form login Admin

- Dashboard Admin

Dashboard admin digunakan untuk memantau data booking servis, mengelola layanan, menugaskan mekanik, dan mengirim invoice kepada pelanggan.



Nomor Booking	Tanggal	Motor	Layanan	Status	Aksi
BK-20201114-0005	14 Jan 2020 10:34	AB 330204 CD Honda Beat	Cuci Motor	Selesai	Kirim Invoice
BK-20201114-0002	14 Jan 2020 10:43	AB 330204 CD Honda Beat	Ganti Ban	Selesai	Kirim Invoice
BK-20201114-0001	14 Jan 2020 10:43	AB 330204 CD Honda Beat	oil	Selesai	Kirim Invoice
BK-20201230-7621	30 Dec 2020 10:10	AB 330204 CD Honda Beat	Ganti Kampas Rem	Selesai	Kirim Invoice
BK-20201230-7603	30 Dec 2020 10:10	AB 330204 CD Honda Beat	Cuci Motor	Selesai	Kirim Invoice
BK-20201230-0509	30 Dec 2020 10:34	AB 330204 CD Honda Beat	Cuci Motor	Pending	Kirim Invoice

Gambar halaman Dashboard Admin

- Halaman Admin Kelola user

Halaman Kelola User digunakan oleh admin untuk mengelola data pengguna yang terdaftar dalam sistem, khususnya pelanggan, dan menambahkan data mekanik. Melalui halaman ini, admin dapat memantau, menambah, mengubah, dan menghapus data user guna memastikan sistem berjalan dengan tertib dan data tetap valid.

ID	Username	Nama	Email	Role	Telepon	Status	Aksi
10	mekanikREM	Basuki Rety	mekanik@gmail.com	Admin	-	Aktif	Detail Edit Hapus
9	mekanikKuspernel	Alexand satrio	satrio@gmail.com	Admin	-	Aktif	Detail Edit Hapus
8	mekanikban	pro ban	mekanikban@gmail.com	Admin	-	Aktif	Detail Edit Hapus
6	admin		admin@motofix.com	Admin	-	Aktif	Detail Edit Hapus
1	customer	berhan	-	Customer	-	Aktif	Detail Edit Hapus

Total 5 user

Gambar halaman admin Kelola user

- Halaman admin Kelola layanan servis

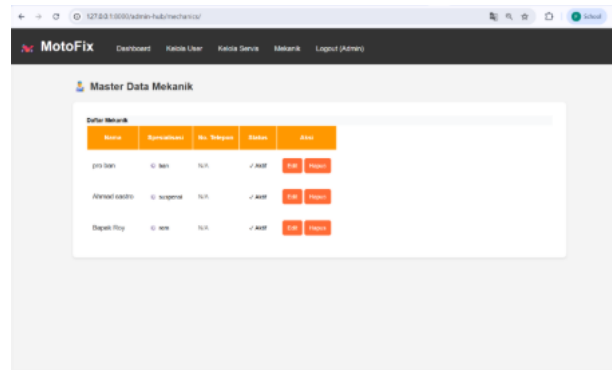
Antarmuka manajemen data master layanan bengkel. Fitur ini memungkinkan admin untuk memperbarui daftar jenis servis, mengatur harga, serta deskripsi pengerjaan secara dinamis guna menjaga akurasi informasi bagi pelanggan.

Layanan	Detail Harga & Waktu	Status	Aksi
Maintenance Ban Pengerjaan ban untuk kendaraan beroda dua	Harga: Rp 75000 Estimasi: 20 menit	Aktif	Edit
Ganti Ban Pengerjaan ban untuk kendaraan beroda dua	Harga: Rp 50000 Estimasi: 20 menit	Aktif	Edit
Ganti Ban Pengerjaan ban dengan kualitas terbaik	Harga: Rp 30000 Estimasi: 20 menit	Aktif	Edit
Ganti Filter Oli Pengerjaan filter oli untuk motor	Harga: Rp 20000 Estimasi: 20 menit	Aktif	Edit
Ganti Kampas Rem Pengerjaan kampas rem untuk kendaraan	Harga: Rp 120000 Estimasi: 45 menit	Aktif	Edit

Gambar halaman admin Kelola layanan servis

- Halaman admin Kelola mekanik

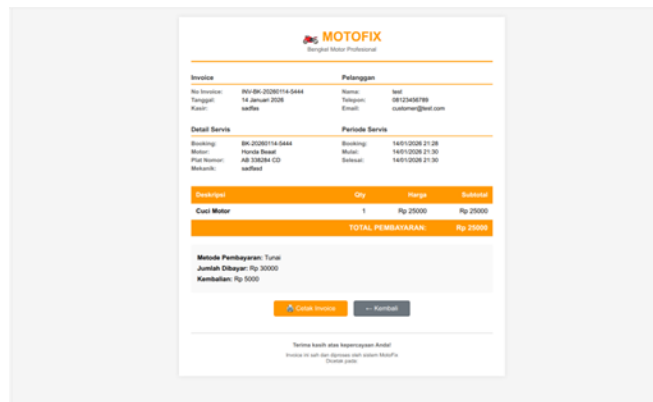
Antarmuka Halaman Admin Kelola Mekanik, yang berfungsi sebagai pusat manajemen data staf teknis bengkel. Halaman ini memungkinkan Admin untuk menambah, melihat, mengedit, atau menonaktifkan akun mekanik, serta memantau status keaktifan dan spesialisasi masing-masing mekanik. Fitur ini krusial untuk memastikan ketersediaan sumber daya manusia dalam proses pengalokasian tugas servis.



Gambar halaman admin Kelola mekanik

- Halaman invoice (admin mencetak invoice)

Halaman Invoice, dokumen digital yang menyajikan rincian biaya jasa, suku cadang, dan total transaksi. Halaman ini memuat nomor invoice, identitas kendaraan, serta status pembayaran sebagai bukti transaksi sah dan arsip finansial bengkel.



Gambar halaman admin cetak invoice

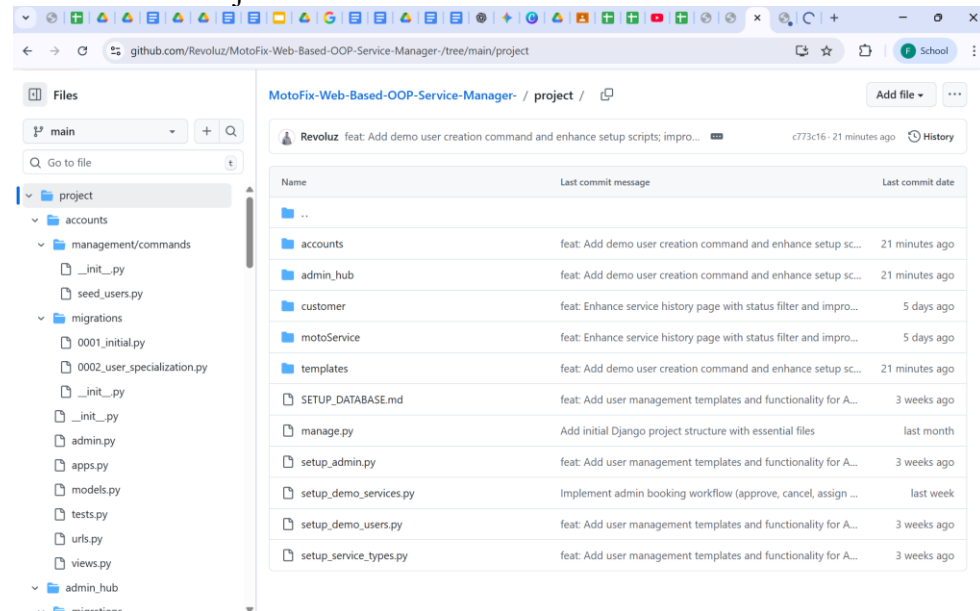
h. Penjelasan screenshot status unggah skrip di Gitlab/Github hingga proyek final.

- Gambaran Umum Struktur Proyek

Proyek MotoFix disusun menggunakan framework Django dengan pola arsitektur berbasis *modular application*. Struktur folder dibagi berdasarkan fungsi aplikasi agar sistem mudah dikembangkan, dirawat, dan dipahami.

Struktur utama proyek berada di dalam folder `project/` yang berisi beberapa aplikasi utama, folder `template`, serta file konfigurasi.

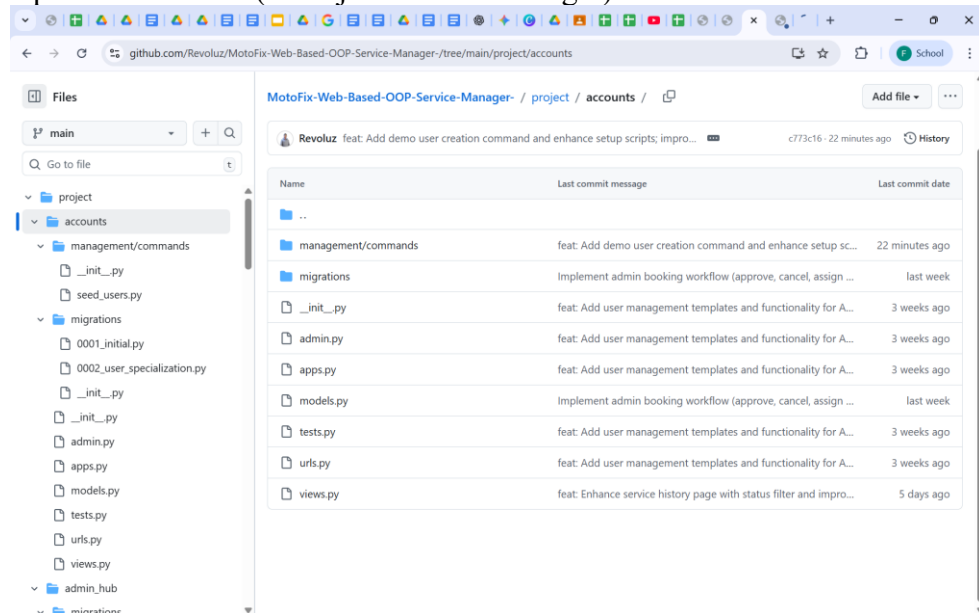
- Folder utama Project



Folder: `project/`

Folder ini merupakan direktori utama yang berisi seluruh kode sumber aplikasi MotoFix. Di dalamnya terdapat beberapa aplikasi Django, konfigurasi sistem, serta template tampilan antarmuka.

- Aplikasi Accounts (Manajemen User & Login)



Folder: project/accounts/

Folder ini berfungsi untuk mengelola seluruh proses autentikasi dan manajemen user.

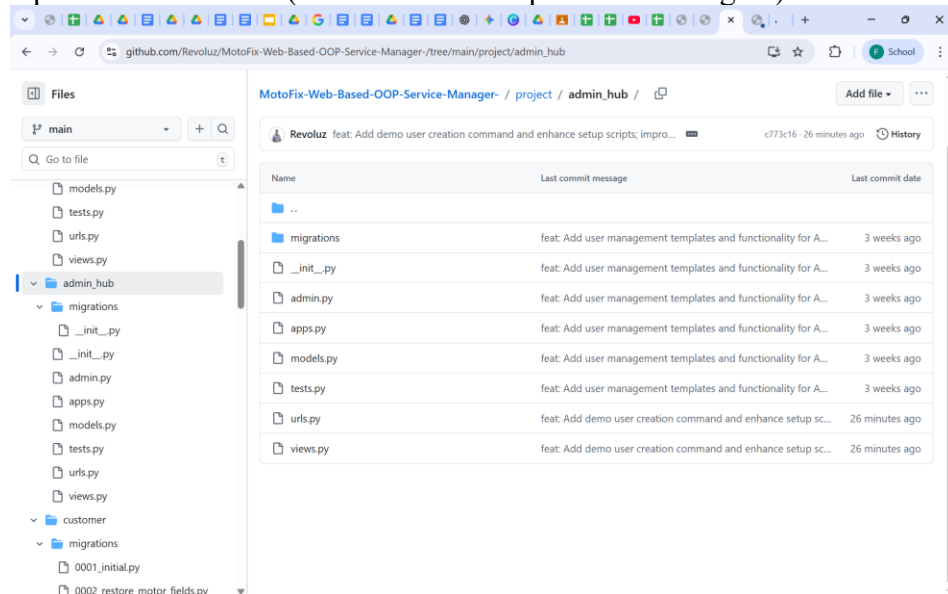
Isi Penting :

- models.py → mendefinisikan model User (customer, mechanic, cashier, admin)
- views.py → menangani proses login, register, dan validasi user
- urls.py → routing halaman login dan register
- management/commands/seed_users.py → script untuk membuat user demo otomatis
- Migrations/ → file migrasi database untuk tabel user

Fungsi utama:

- Login dan autentikasi user
- Penyimpanan data user
- Pengaturan role (pelanggan, mekanik, kasir, admin)

- Aplikasi Admin Hub (Panel Admin & Operasional Bengkel)



Folder: project/admin_hub/

Folder ini merupakan pusat pengelolaan sistem oleh admin dan staff bengkel.

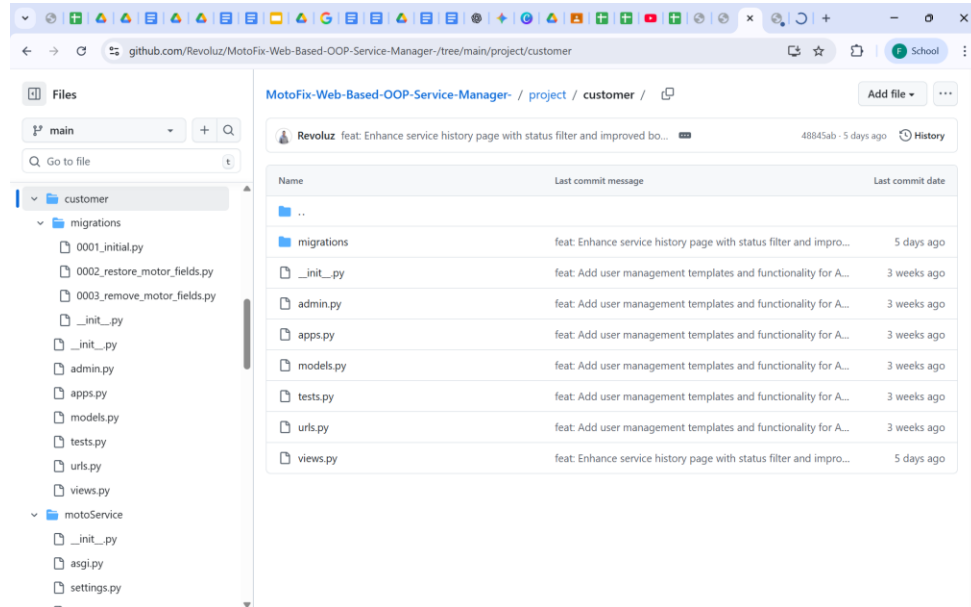
Isi penting:

- views.py → berisi seluruh logika kelola booking, assign mekanik, invoice, user, dan layanan
- urls.py → routing seluruh halaman admin
- models.py → relasi data tambahan jika ada
- migrations/ → migrasi database admin

Fungsi utama :

- Dashboard admin
- Kelola booking servis
- Approve & cancel booking
- Tugaskan mekanik
- Proses pembayaran dan invoice
- Kelola user dan mekanik
- Kelola layanan servis

- Aplikasi Customer (Fitur Pelanggan)



Folder: project/customer/

Folder ini menangani seluruh aktivitas pelanggan.

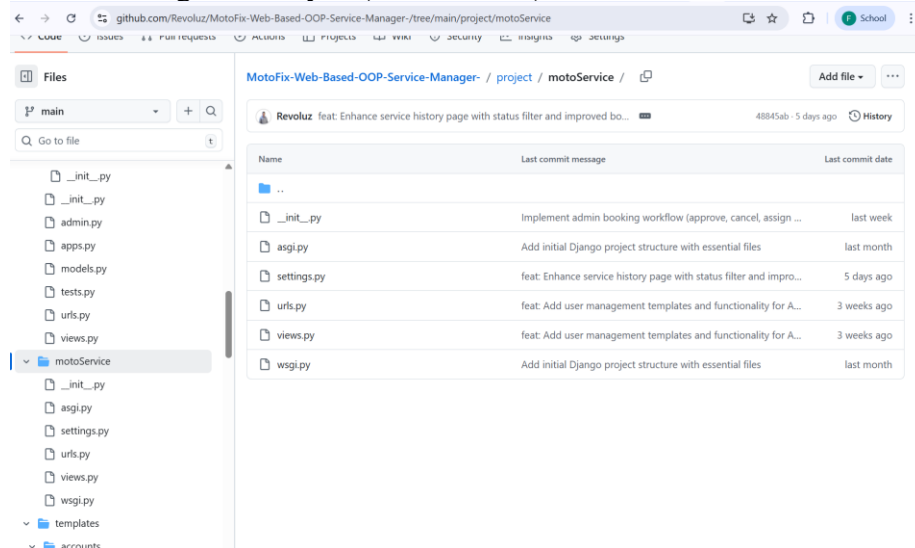
Isi penting:

- models.py → model Motor, ServiceBooking, Invoice, SparePart
- views.py → proses tambah motor, booking servis, riwayat servis
- urls.py → routing halaman pelanggan
- migrations/ → migrasi database motor, booking, invoice

Fungsi utama:

- Tambah dan edit data motor
- Booking servis (online)
- Melihat status servis
- Melihat riwayat servis
- Melihat invoice digital

- Folder Konfigurasi Proyek (motoService)



Folder: project/motoService/

Folder ini merupakan inti konfigurasi Django.

Isi file penting:

- settings.py → konfigurasi database, aplikasi, template, static file
- urls.py → routing utama sistem
- wsgi.py dan asgi.py → konfigurasi server deployment

Folder ini mengatur bagaimana seluruh aplikasi terhubung dalam satu sistem

- Folder Templates (Tampilan Antar muka GUI)

Folder: project/templates/

Folder ini berisi seluruh file HTML sebagai tampilan antarmuka sistem.

i. Analisis pengerjaan proyek (tinjauan dari sisi waktu, ketercapaian spesifikasi, biaya yang dibutuhkan, kendala, tantangan masa depan, dan atau lain-lain jika ada)

1. Tinjauan dari sisi waktu

Pengerjaan proyek sistem informasi servis motor MotoFix dilakukan dalam beberapa tahapan, yaitu:

- a. Tahap perancangan konsep dan pembagian tugas kelompok
- b. Tahap perancangan database dan model sistem
- c. Tahap implementasi fitur utama (login, booking, dashboard, admin panel)
- d. Tahap pengujian dan perbaikan bug
- e. Tahap finalisasi dan dokumentasi proyek

Secara umum, waktu pengerjaan proyek berjalan sesuai dengan jadwal yang direncanakan. Setiap tahap dapat diselesaikan secara bertahap hingga sistem mencapai kondisi stabil dan siap digunakan. Proses pengembangan juga didukung dengan penggunaan GitHub sehingga perubahan kode dapat tercatat dengan baik.

2. Tinjauan dari Sisi Ketercapaian Spesifikasi

Berdasarkan spesifikasi awal yang telah ditentukan, sistem berhasil memenuhi hampir seluruh kebutuhan fungsional.

Spesifikasi yang berhasil dicapai:

- a. Fitur Pelanggan (Customer)
 - Login pelanggan
 - Menambah data motor
 - Melakukan booking servis (online)
 - Melihat status servis
 - Melihat riwayat servis
 - Melihat invoice digital
- b. Fitur Admin
 - Login admin
 - Melihat dashboard operasional
 - Mengelola data booking
 - Approve dan cancel booking
 - Menugaskan mekanik
 - Memulai dan menyelesaikan servis
 - Mengelola data mekanik
 - Mengelola data user
 - Mengelola jenis layanan servis
 - Membuat dan melihat invoice

Seluruh fitur utama sesuai spesifikasi berhasil diimplementasikan dengan baik. Sistem telah mampu menjalankan alur operasional bengkel secara lengkap dari sisi pelanggan hingga admin.

3. Tinjauan dari Sisi Biaya yang Dibutuhkan

Dalam pengerjaan proyek ini, biaya yang dibutuhkan relatif minimal karena:

- a. Framework Django bersifat open source dan gratis
- b. Database SQL digunakan sebagai database default
- c. Platform GitHub digunakan sebagai media penyimpanan kode gratis
- d. Pengembangan dilakukan menggunakan perangkat pribadi

Biaya utama hanya berupa:

- Biaya listrik dan internet
- Waktu dan tenaga pengembang

Dengan demikian, proyek ini tergolong efisien dari sisi biaya pengembangan.

4. Kendala yang Dihadapi Selama Pengerjaan

Beberapa kendala yang ditemui selama proses pengembangan antara lain:

- a. Pemahaman Framework Django
Pada tahap awal, terdapat kesulitan dalam memahami struktur proyek Django, terutama pada:
 - Pengaturan routing URL
 - Relasi antar model
 - Penggunaan template dan context
- b. Pengelolaan Role User
Pengaturan hak akses berdasarkan role (customer, mechanic, cashier, admin) membutuhkan perancangan yang cukup teliti agar setiap user hanya dapat mengakses fitur sesuai perannya.
- c. Integrasi Antar Fitur
Menghubungkan fitur booking, penugasan mekanik, status servis, dan invoice membutuhkan pengujian berulang agar alur berjalan dengan benar.
- d. Penanganan Error dan Validasi
Beberapa error muncul saat:
 - Proses migrasi database
 - Validasi input form
 - Transisi status booking

Namun kendala tersebut dapat diatasi melalui proses debugging dan perbaikan bertahap.

5. Tantangan Pengembangan di Masa Depan

Beberapa tantangan yang mungkin dihadapi jika sistem dikembangkan lebih lanjut antara lain:

a. Skalabilitas Sistem

Jika jumlah pengguna dan data meningkat, sistem perlu:

- Menggunakan database yang lebih kuat (MySQL / PostgreSQL)
- Optimasi query database
- Pengelolaan server yang lebih baik

b. Keamanan Sistem

Pengembangan lanjutan perlu memperhatikan:

- Enkripsi data sensitive
- Pengamanan session dan cookie
- Proteksi terhadap serangan SQL Injection dan CSRF

c. Pengembangan Fitur Tambahan

Beberapa fitur yang dapat dikembangkan di masa depan:

- Notifikasi email atau WhatsApp
- Sistem pembayaran online
- Dashboard khusus mekanik
- Laporan keuangan otomatis
- Grafik statistik servis

6. Kesimpulan

Secara keseluruhan, proyek sistem informasi servis motor MotoFix berhasil dikembangkan dengan baik sesuai spesifikasi yang direncanakan. Sistem telah memenuhi kebutuhan utama pelanggan dan admin, berjalan stabil, serta memiliki potensi besar untuk dikembangkan lebih lanjut di masa depan. Proyek ini menjadi bukti penerapan konsep pemrograman berorientasi objek dalam pengembangan sistem informasi berbasis web secara nyata.